

C200 04F1F9A05934062F : 50
C208 0207246712380843 : 29
C210 3AA642262D0BF1C7 : 38
C218 F1AFF1D279C3AA78 : C1
C220 C372A0DB79C3A978 : 0D
C228 C52080DB415903FF : DC
C230 D0DAD2DA5A2B05BD : 9D
C238 59586B02426B0239 : 06
C240 04448C707159BD44 : 3F
C248 59DA8346DA80C73B : 58
C250 193703F702FFFE1EE : 2A
C258 020134023B348306 : 31
C260 5703FF45532F0786 : AD

C3E8 004D442F024579C5 : 45
C3F0 0578C4A0030090C2 : 36
C3F8 452B03C3DA79C503 : 51
C400 78C3723467003807 : 87
C408 4EC84EFF4EFF6B08 : 23
C410 7CC58F2F0C79C3A9 : F0
C418 10F80056003B1F11 : C9
C420 401F79C2C010C600 : 30
C428 56009F1F79C3AA78 : 72
C430 C37292DB78C37234 : 83
C438 92DB80DB10000002 : DA
C440 51FFB11F0241FFB1 : 13
C448 1F2F0E79C3A9F1C7 : F9

C450 5000F10270C3A470 : 3E
C458 0000000000000000 : 00
C460 0000000000000000 : 00

C450 5000F10270C3A470 : 3E
C458 0000000000000000 : 00
C460 0000000000000000 : 00

SHARP PC-1245/51/55 SC61860 PiO 1986 年 8 月号

PC-インタープリタを読む

976 バイトの SC61860 マシン語を逆アセンブルする

PC-INTERPRETER

おりぐち

C330 7EC4DD07D4392C07 : 00
C338 DB38F5679538C267 : 65
C340 C538E367A138D267 : 59
C348 D77EC47B67127EC4 : 4F
C350 BD67D57EC4C967D1 : 3C
C358 7EC4D867D638F067 : E6
C360 AC38F56789280AF1 : EC
C368 C778C5D0F1D22C3B : FE
C370 2C0778C50090DB37 : 12
C378 67517FC4F1676B7D : 3B
C380 C4F17435DA246731 : F4
C388 280C10C391DA5210 : D4
C390 C68657DA04052467 : 11
C398 3438020510C3A6DA : C6
C3A0 5278C50010C68890 : 7D
C3A8 530524671D7EC30E : 4F
C3B0 433BAB0579C30E00 : 78
C3B8 67392A0567352A02 : 97
C3C0 376735280978C520 : 61
C3C8 904479C505673628 : DC
C3D0 0978C520904579C5 : 79
C3D8 056737281578C520 : 3D

C310 50472D1079C30800 : 0E
C520 2467AF7EC4A867AE : 39
C528 7EC2270524678A28 : A9
C530 0CF1C7FF4490DBF1 : 63
C538 D290DB3767122804 : 19
C540 240437674B394967 : FA
C548 1F38020579C5A000 : 3C
C550 042467317EC56F05 : 77
C558 DB3778C54090DB37 : 31
C560 2575517FC4F1671A : A0
C568 7DC4F1A0DB708610 : B3
C570 C5755310C6885524 : 64
C578 6731390C05DB3700 : F4
C580 79C2DD3778C2E402 : 6F
C588 58325B4234D03778 : DA
C590 C2E4025832950230 : F9
C598 DB379E60079C2CA : 15
C5A0 042467572A286740 : DF
C5A8 3A247540670A3A03 : C1
C5B0 7507A0DB2567577D : 57
C5B8 C4F14D67407FC4F1 : DD
C5C0 7540670A3A037507 : DF

はじめに

Fan_PC-1251 氏 (https://x.com/pio1986_10) の「PC-インタープリタ」(PiO 1986 年 8 月号 pp.169-171) は、PC-1245/51/55 用の BASIC プログラムを BASIC より 5~7 倍速く実行するとされる、976 バイトのマシン語プログラムである。本書では記事のダンプリストを転記して逆アセンブルし、プログラムの解析 (パース)、GOTO、IF/NOT、FOR/NEXT、表示、音、エラー処理の仕組みを図で示す。

SC61860 の命令仕様は資料ごとに食い違いがあるため、複数の資料と PC-1251 の ROM の処理を突き合わせ、このプログラム自身の書き方も証拠にして解釈を決めた。

マシン語に慣れていない人は、まず 1.1 節の基礎知識に目を通してほしい。第 1 章と第 2 章でしくみを説明し、第 3 章「ここが凄い!」で工夫している箇所を図入りで紹介する。第 4 章では別の書き方を試して効果を測り、第 5 章で検証の結果と未解決の点をまとめた。各章の終わりには、SC61860 の癖や由来にまつわる話をコラムとして置いた。付録に注釈付きの逆アセンブル・リストを収めた。

命令は POPCOM 1984 年 9~11 月号の連載「PC-1250/1251 マシン語講座」の BASIC 風命令記号で書き、括弧内にシャープのオリジナルニーモニックを添えた (1.2 節)。私が初めてマシン語を学んだのがこの POPCOM の講座だった。A=(+X) のような書き方は、ニーモニックを知らなくても命令の動きを思い浮かべやすいので、本書でも使わせてもらうことにした。

サイクル数は utz82 の命令表による概算である。「実測」とあるのは、実 ROM を載せたエミュレータで命令のサイクル数を数えた値で、実機を時計で測った値ではない。

逆アセンブル、ROM の調査、エミュレータの作成と測定、本書の図版と組版には、Anthropic の Claude を用いた。

目次

第1章	準備	1
1.1	読むための基礎知識	1
1.2	命令の書き方	3
1.3	レジスタとフラグ	4
1.4	対象と進め方	7
1.5	命令仕様の確認	8
1.6	メモリの使い方	10
1.7	BASIC とのつながり	11
コラム	レジスタはメモリの中にある	12
コラム	上位が先か、下位が先か	13
第2章	しくみ	15
2.1	実行の骨組み	15
2.2	式の解析	16
2.3	略記法のしくみ	19
2.4	GOTO	20
2.5	IF と NOT	20
2.6	FOR と NEXT	21
2.7	表示	21
2.8	時間と音	23
2.9	エラーと BREAK	24
コラム	戻り番地と戻り値	24
コラム	前向きと後ろ向きで別の命令	25
第3章	ここが凄い!	27
その1	BASIC に「間借り」する	27
その2	命令のオペランドを書き換えて番地を作る	28
その3	次の命令の1バイトを定数として読む	28
その4	書けない ROM に書いて時間を稼ぐ	29
その5	よその JP を借り、フラグごと飛び込む	29

その 6	比較結果を 3 値にして、ポロー 1 本で判定する	30
その 7	読み飛ばし 1 つで ELSE を作る	30
その 8	16 進は 2 文字目から読む	31
その 9	偏らない乱数	31
その 10	1/256 秒に合わせた WAIT	31
その 11	掛け算は小さい方を回数にする	32
その 12	BASIC のエラー表示を乗っ取る	32
コラム	割り込みのない CPU	33
コラム	非公開だった命令表	33
第 4 章	別の書き方と比べる	35
4.1	振り分けの順序	35
4.2	シフトと加算による掛け算	36
4.3	PRINT の書き込み位置の計算	36
4.4	ブロック転送命令の使いどころ	37
4.5	GOTO の行探しと行頭表	37
4.6	IF のエラー番号	38
コラム	電卓の血筋	38
コラム	同じ CPU を積んだ機種	38
第 5 章	検証と残った疑問	39
5.1	記事と食い違う点・未確認の点	39
5.2	検証	39
付録 A	命令一覧	41
付録 B	注釈付き逆アセンブル・リスト	44
参考文献		57

第 1 章

準備

1.1 読むための基礎知識

本書にはマシン語の用語がたくさん出てくる。初めての人のために、よく使うものをここにまとめておく。すでに知っている人は読み飛ばしてかまわない。

16 進法とバイト

コンピュータの中の値は 0 と 1 の並び (2 進法) だが、そのまま書くと長くて読みにくい。そこで 2 進法の 4 桁 (4 ビット) をまとめて 1 文字で表す 16 進法を使う。0~9 のあとに A~F (10~15) を使い、たとえば 2 進法の 1100 0011 は 16 進法で C3 になる。16 進法の 1 桁がちょうど 2 進法の 4 桁に対応するので、値を簡潔に書きながら、各ビットの状態もすぐ読み取れる。10 は 10 進法の 16、FF は 255 である。本書では 16 進法の値を C300 のように等幅の字で書く。1 バイトは 8 ビット、つまり 16 進法で 2 桁で、00~FF の 256 通りの値を持つ。このインタープリタの数値はすべて 1 バイトである。

番地 (アドレス)

メモリの 1 バイトごとに振られた通し番号。PC-1245 では 0000~FFFF の番地に ROM、RAM、液晶が割り当てられている (図 1.6)。

命令、オペコード、オペランド

CPU が実行する 1 つ 1 つの命令は、種類を表す 1 バイト (オペコード) と、必要に応じてそれに続く値 (オペランド) でできている。78 C3 72 は「CALL(78) で C372 番地を呼ぶ」という 3 バイトの命令である。CALL のような英字の名前はニーモニックと呼ぶ。

レジスタとポインタ

CPU が計算に使う小さな記憶場所。SC61860 の A や B は 1 バイト、X や Y は 2 バイトで、X や Y には番地を入れて「そこを読み」という指差し (ポインタ) として使う。{A=(+X)} (IXL) は「X を 1 進めてから、その番地の値を A に読む」命令で、X は常に最後に読んだバイトを指す。

フラグ、キャリー、ボロー

計算の結果を 1 ビットで覚えておく印。ゼロフラグ (Z) は結果が 0 のとき 1 になる。キャリーフラグ (C) は、足し算で桁があふれたときと、引き算で桁が足りなくなったときに 1 になる。引き算で桁が足りなくなることボロー (桁借り) という。1 バイトの引き算 00 - 01 は、上の桁

から借りて FF になり、キャリーフラグが 1 になる (図 1.1)。

比較と条件分岐

A-n (CPIA) などの比較命令は、引き算をして答えを捨て、フラグだけを残す。続く分岐命令が、そのフラグを見て行き先を変える。JRP Z,n (JRZP) は Z が 1 のとき、JRP C,n (JRCP) は C が 1 のときに分岐する。IF 文やトークンの判定は、すべてこの組み合わせで書かれている。

ジャンプ、サブルーチン、スタック

ジャンプは次に実行する番地を変える命令で、番地をそのまま書く絶対ジャンプ (JP、3 バイト) と、今の位置からの距離を書く相対ジャンプ (JR、2 バイト) がある。サブルーチンは CALL で呼び、RETURN (RTN) で戻る。呼ぶときに「戻り番地」をスタック (後に入れたものから取り出す記憶場所) に積み、戻るときに取り出す (図 1.2)。

サイクル

命令の実行にかかる時間の単位。PC-1245 では 1 サイクルが約 $5.2 \mu\text{s}$ (192 kHz) で、A に値を入れる A=n (LIA) は 4 サイクル、CALL は 8 サイクルかかる。

中間コードとトークン

BASIC はプログラムを入力されたままの文字では保存せず、PRINT や GOTO のようなキーワードを 1 バイトの記号 (トークン) に置き換えて保存する。これを中間コードという。このインタープリタが読むのはこの中間コードである。

自己書き換え

実行中のプログラムが、自分自身の命令のバイトを書き換えること。プログラムが RAM にあるから使える方法で、第 3 章その 2 で詳しく扱う。

トランポリン

飛び先の番地が実行してみるまで決まらないときに、その番地を書き込んだ JP 命令を RAM に 1 つ作り、そこを経由して飛ぶ方法。いったんそこへ跳んで、すぐ本当の行き先へ跳ね返されるので、こう呼ばれる。SC61860 には「レジスタに入っている番地へ飛ぶ」命令 (間接ジャンプ) がないため、BASIC の CALL &C300 はこの方法で実現されている (図 1.3、1.7 節)。

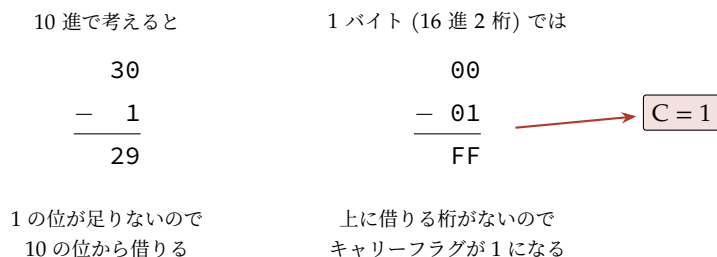


図 1.1 ボロー (桁借り)。A=A-1 (DECA) や A-n (CPIA) の結果が負になるときにキャリーフラグが立つ

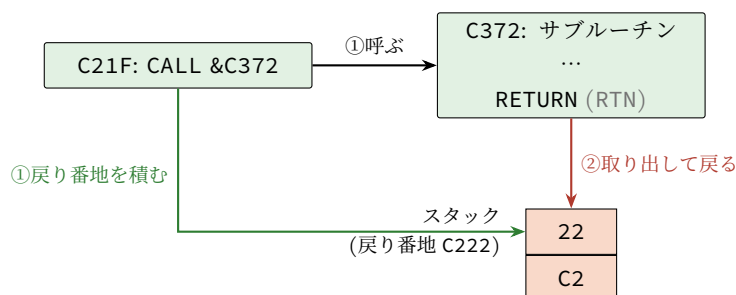


図 1.2 サブルーチンの呼び出しと戻り番地。CALL は次の命令の番地 (C222) をスタックに積んでから飛ぶ



図 1.3 トランポリン。BASIC は固定の番地 C6DB を呼び、そこに書いた JP が本当の飛び先へ跳ね返す

1.2 命令の書き方

本書では命令を、POPCOM 1984 年 9～11 月号の連載「PC-1250/1251 マシン語講座」(以下、POPCOM の講座) の BASIC 風命令記号で書き、括弧内にシャープのオリジナルニーモニックを添える。たとえば{A=(+X)} (IXL)、A-&34 (CPIA) のように書く。

1.2.1 BASIC 風命令記号

POPCOM の講座は、米国で販売されていたシャープの英文マニュアル (SHARP POCKET COMPUTER PC-1250/1251(1250A) MACHINE LANGUAGE REFERENCE MANUAL) を編集部が入手し、それをもとに書かれた。マニュアルには独特のニーモニックが使われているが、初めて見る人にはなじみにくいので、編集部が「マシン語の働きが連想できるような」BASIC 風の記号を作った。表 1.1 に記号の決まりをまとめる。

ニーモニックを覚えていなくても、A=(P) (LDM) や A↔(P) (EXAM) なら何をする命令か見当がつく。一方で、シャープのニーモニックはどの資料でもほぼ共通なので、ほかの資料やエミュレータと照らし合わせるときの手がかりになる。そこで両方を並べることにした。

1.2.2 本書での補い方

講座の記号に次の 4 点を補った。

表 1.1 BASIC 風命令記号の決まり (POPCOM の講座の表 4 による)

記号	意味
A、P、DP など	レジスタそのもの (DPL は DP の下位 8 ビット)
(P)、(DP)	P が指す内部 RAM の内容、DP が指す (外部) メモリの内容
(P+1,P)、(B,A)	2 バイトを組にした 16 ビット値 (左が上位)
左辺=右辺、左辺↔右辺	代入と交換
A-n、(P)-A	比較。引き算をしてフラグだけを変え、答えは捨てる
A.AND.n	AND の結果で Z フラグだけを変える (テスト)
&	16 進法の値
{ }	1 命令で順に行う複数の動作。{A=(+X)}は「X を 1 進め、DP にも入れ、その番地を A に読む」
[],I、[],J	ブロック処理。I または J の値に 1 を足したバイト数だけ繰り返す (添字がなければ I)
JRP、JRM、JP	前への相対ジャンプ、後ろへの相対ジャンプ、絶対ジャンプ。C、NC、Z、NZ、が付くと条件付き

- 相対ジャンプと LOOP の行き先は、講座ではオペランドの値 (距離 n) で書くが、本書では飛び先の番地からラベルで書く。
- 講座に載っていない命令は 56 の 1 つだけで、 $A=(PC+1)$ (READ) と書く (本書独自の表記。付録では $\dagger$ を付けた)。
- バイト列の上に命令を示す図では、BASIC 風の記号の下にニーモニックを置く。幅が足りない図ではニーモニックだけを書く。
- BASIC 風の記号とニーモニックが同じ語のとき (CALL、JP、PUSH、LOOP、WAIT など) は括弧を省く。

このプログラムが使う命令は 73 種類、延べ 540 命令で、56 を除く 72 種類はすべて講座の命令表に載っている。付録の表 A.1 に一覧を示した。講座の表 9 に並ぶシャープのニーモニックは、utz82 の表の名前と一致する。

1.3 レジスタとフラグ

SC61860 のレジスタは 2 種類に分かれる (POPCOM の講座の表 2)。I、J、A、B、X、Y、K~N は内部 RAM の 00~0B 番地にあり、P、Q、R、DP、PC は CPU の中に独立して置かれている。フラグは C と Z の 2 つだけである。図 1.4 に、レジスタとメモリのつながりを示す。

1.3.1 それぞれの役割

表 1.2 に、各レジスタの役割とこのプログラムでの使い方をまとめる。

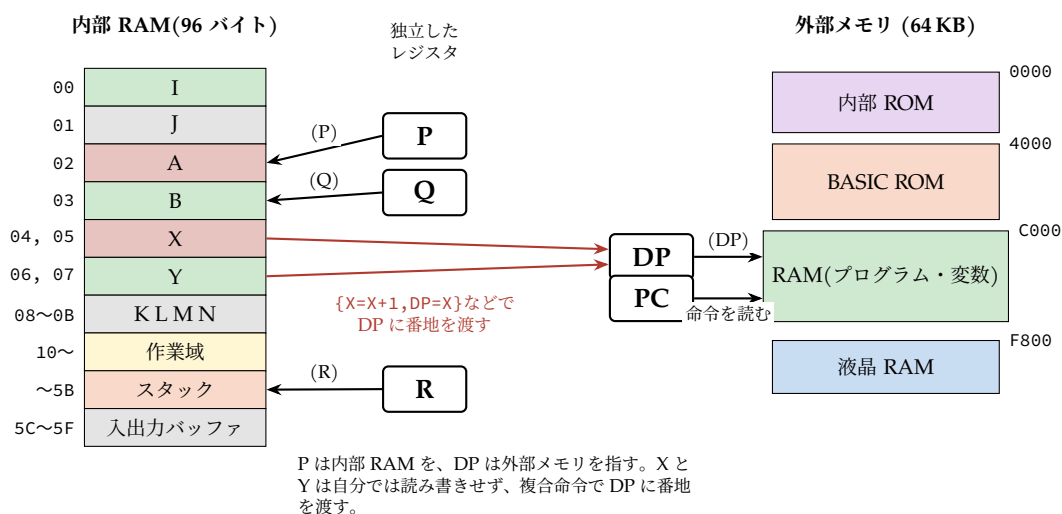


図 1.4 レジスタとメモリのつながり。内部 RAM の 00~0B 番地がレジスタを兼ねる

表 1.2 レジスタの役割 (講座の表 2 をもとに、このプログラムでの使い方を加えた)

名前	場所	ビット	役割	このプログラムでの使い方
A	02	8	アキュムレータ。ほとんどのデータが A を通る	読んだ文字、式の途中の値
B	03	8	補助。(B,A) で 16 ビットにもなる	変数の番地の下位、除算の商など。 A↔B (EXAB) で出し入れする
I, J	00, 01	8	ブロック命令の長さ (-1)	I: 消去するバイト数、RND の $n-1$ 。 J はこのプログラムでは使わない
X	04, 05	16	インデックス (読み出し用)	BASIC プログラム上の実行位置。常に最後に読んだバイトを指す
Y	06, 07	16	インデックス (書き込み用)	PRINT の表示バッファ、SQR と DEG の液晶 RAM
K~N	08~0B	8	汎用	使わない
P	独立	7	内部 RAM のポインタ	ほぼすべての計算。10 番地 (計算結果) や 20 番地 (カーソル) を指す
Q	独立	7	内部 RAM の 2 つ目のポインタ	FOR で X を保存するときだけ
R	独立	7	スタックポインタ	呼び出しと LOOP の回数。エラー時に R=A (STR) で積み直す
DP	独立	16	外部メモリのポインタ	変数・配列・液晶 RAM の読み書き (自己書き換えて設定)
PC	独立	16	次に実行する命令の番地	A=(PC+1) (READ) で次の 1 バイトを読む

1.3.2 振る舞いで押さえておくこと

計算は A と (P) のあいだで行う

レジスタどうしの演算命令はなく、足し算も比較も「A と (P)」「(P) と即値」「A と即値」の形しかない(講座の「マシン語命令の特徴」)。B に入れた値を使うには $A \leftrightarrow B$ (EXAB) で A と入れ替える。このプログラムに $A \leftrightarrow B$ (EXAB) が 12 回、 $A \leftrightarrow (P)$ (EXAM) が 22 回現れるのはこのためである。

レジスタは内部 RAM の番地でもあるので、P でレジスタ自身を指せば、メモリ用の命令をレジスタに使える。画面反転 (SQR、C252) は $P = \&03$ (LP) で B を指し、 $(P) = (P) - A$ (SBM) で「 $B \leftarrow B - A$ 」を計算する。B にはあらかじめ FF を入れてあるので、結果は $FF - A$ 、つまり A の全ビットを反転した値になる。続く $P = \&06$ (LP) と $(P) = (P) + \&04$ (ADIM) は、Y の下位バイト (06 番地) に直接 4 を足している。

P は 7 ビットで、 $P = \ell$ (LP) は 3F までしか届かない

1 バイトの $P = \ell$ (LP) が指せるのは 00~3F である。スタックのある 40 番地以上を指すには 2 バイトの $P = n$ (LIP) が必要。エラー処理で戻り番地 (5A 番地) を書き換えるところだけ $P = \&5A$ (LIP) を使っている。 $A = (P)$ (LDM) などの 1 バイト命令は P を動かさないが、ブロック命令は P を先へ進める。

外部メモリは必ず DP を通す

外部メモリを読み書きする命令は、どれも DP が指す番地を使う。DP を設定する方法は、 $DP = nm$ (LIDP) (3 バイト)、下位だけを変える $DPL = n$ (LIDL) (2 バイト)、X か Y の値を写す複合命令の 3 つしかない。DP だけを 1 増やす命令はない。そのため、変数の番地は $DP = nm$ (LIDP) のオペランドを書き換えて作る (第 3 章その 2)。

X と Y は「進めてから」使う

$\{A = (+X)\}$ (IXL) は X を 1 進め、DP にも同じ値を入れてから、その番地を A に読む。 $\{(+Y) = A\}$ (IYS) も同じ順で書く。講座は「A をメモリーにストアするときは、Y レジスターを使い、逆に (中略) ロードするときは、X レジスターを使っています」と書いている。X は読み、Y は書き、と役割が分かれている。このプログラムは X を実行位置に専用し、ROM の 11C7 と 11D2 でスタックに退避して守る。

論理演算は AND と OR だけ

SC61860 の論理演算は AND と OR の 2 つで、排他的論理和 (XOR) も、全ビットを反転する NOT もない(講座の「論理演算命令」)。上の SQR が $FF - A$ で反転を作っているのはこのためである。FF から引く引き算では桁借りが起きないので、各ビットを反転した結果と一致する。

ブロック命令は I を減らさない

講座の説明では、ブロック命令は I の値を見えない内部のカウンタ d に写してから数える。I 自体は変わらないので、同じ長さの処理を続けるときに設定し直す必要がない。画面消去 (COS、C418) は $I = \&3B$ (LII) を 1 回だけ実行し、 $[(DP) = A]$ (FILD) で液晶の左半分 60 バイトを消

したあと、DPL=&40 (LIDL) で右半分に移って、もう一度 [(DP)=A] (FILD) を実行する。2 回目も I=3B のまま 60 バイトが消える。

R はスタックを下へ伸ばす

スタックは内部 RAM の 5B 番地から小さい番地へ伸びる。PUSH は R を 1 減らしてから書き、CALL は戻り番地の 2 バイトを積む。LOOP はスタックの一番上を回数として減らす。CALL &C300 で入った直後の R は 58 である。インタプリタはエラーや BREAK のとき、R=A (STR) で R をこの 58 に戻し、途中で積んだものをまとめて捨てる。

シャープのニーモニックに (R)=0 (LEAVE) という命令がある。スタックの一番上を 0 にする命令で、次の LOOP が 1 を引くと桁借りが出て、ループを抜ける。名前のとおり、ループを途中で打ち切るのに使える。このプログラムは使っていない。

フラグは C と Z だけで、変わらない命令が多い

代入、交換、P=l (LP)、PUSH、POP などはフラグを変えない。比較の結果を残したままデータを動かせる。別のルーチンの JP Z,nm (JPZ) にフラグごと飛び込む書き方 (第 3 章その 5) も、この性質があるから使える。一方、A=A-1 (DECA) のような増減命令は C と Z を変える。IF の判定はこのボローを使う (第 3 章その 6)。SET C (SC) と RESET C (RC) は、C と同時に Z も 1 にする。

増減命令は Q も変える

講座の命令表では、A=A+1 (INCA) などの「変化レジスター」に Q が挙がっている。レジスタが内部 RAM の番地なので、増減した番地 (A なら 02) が Qに残ると読める。このプログラムは Q を使う直前に Q=&04 (LIQ) で設定し直しているので、影響を受けない (コラム「レジスタはメモリの中にある」)。

1.4 対象と進め方

1.4.1 記事の概要

PC-インタプリタは、BASIC で書いたプログラムを独自の文法で解釈して実行する。使える命令は PRINT、IF~NOT、GOTO、FOR~NEXT、BEEP、WAIT など十数個で、数値はすべて 1 バイト (16 進 2 桁) である。マシン語は C200~C5CF の 976 バイトで、BASIC から CALL &C300 で起動する。

1.4.2 作業の流れ

1. 記事のリスト 1 (1 行 8 バイト + チェックサム、全 122 行) をスキャン画像から転記した。チェックサムは 8 バイトの単純和の下位 8 ビットで、全行で一致した。C568 の行には手書きの修正があり、修正後の値で一致する。
2. SC61860 は 1~4 バイトの可変長命令なので、C300 と記事の表 1 の各アドレスから分岐をたどって命令の区切りを決めた (再帰下降)。どこからも実行されない 30 バイトは旧版の残骸と見られる。

3. 命令の意味は複数の資料を比べて決めた (1.5 節)。
4. ROM ルーチンの役割は、PC-1251 の ROM を逆アセンブルして確かめた (1.7 節)。
5. 解釈を実装したエミュレータで、記事のサンプルプログラムを含むテストを実行した (5.2 節)。

1.5 命令仕様の確認

1.5.1 参照した資料

utz82

[utz82/SC61860-Instruction-Set](#)。命令表とサイクル数。本書の基準。

MAME

[mamedev/mame](#) の `scops.hxx`(実行部) と `scdasm.cpp`(逆アセンブラ)。PC-1250 系のメモリマップ、LCD の並び、クロック (192 kHz) も参照した。

PockEmul

[matsumo/PockEmul](#) の `src/cpu/sc61860.cpp`。メモリマップ図も参照した。

oldsce

[hiroog/sc61860emu](#)(小笠原博之、1994 年)。当時の手作業による解析をもとにした CPU エミュレータ。

POPCOM

「PC-1250/1251 マシン語講座」(POPCOM 1984 年 9~11 月号)。シャープの英文マニュアルにもとづく命令表とサイクル数、レジスタ、入出力ポート、メモリマップの解説。本書の命令の書き方もこれによる (1.2 節)。

1.5.2 資料間で食い違う命令

このプログラムが使う命令のうち、資料で扱いが分かれるものを表 1.3 にまとめた。いずれも、プログラムの書き方に合うのは一つの解釈だけである。

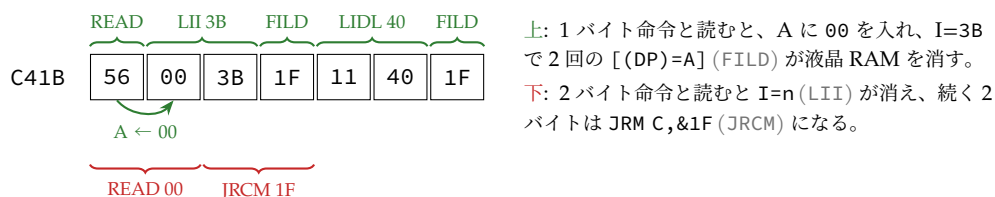


図 1.5 COS(画面消去) の先頭。56 の長さで命令の区切りが変わる

この節のまとめ

utz82 の表はおおむね正確で、POPCOM の講座 (シャープのマニュアルによる) ともよく合う。ただし、どちらにも LOOP の終了時 POP は書かれておらず、utz82 には TEST のビッ

表 1.3 資料による命令の扱いの違いと、本書の採用

命令	資料の記述	本書の採用と根拠
READ(LDPC, 56)	utz82・oldsce・MAME 逆アセンブラ: 1 バイト命令で、次の番地の値を A に読む (そのバイトは消費しない)。MAME 実行部: 2 バイト命令で (P) に即値。 PockEmul: 2 バイト命令で A に即値。 POPCOM: 命令表に載っていない。	1 バイト命令を採用。C41B と C428 は 56 00 3B の並びで、2 バイト命令だと I=n (LII) が実行されず画面消去・変数クリアが働かない (図 1.5)。
LOOP n(2F)	utz82: (R) を 1 減らし、ボローがなければ分岐 (スタックの扱いは記述なし)。 PockEmul・oldsce: 終了時にカウンタを POP する。MAME: 分岐するたびに POP する。POPCOM: (R) を 1 減らして C=0 なら分岐。「ループを出ると (R) は FF」とだけ書き、POP には触れていない。	終了時 POP を採用。SQR の 2 重ループ (PUSH を 2 回、POP なし) や PRINT のループは、終了時に 1 つ取り除かれしないとスタックが崩れる。
WAIT n(4E)	utz82・POPCOM: $6 + n$ サイクル。 MAME: $9 + 2n$ サイクル。PockEmul: ほぼ $6 + n$ 。	utz82 を採用。192 kHz で計算すると WAIT の 1 単位が約 3.9 ms になり、記事の「 $n/256$ 秒」(3.9 ms) と合う。MAME の値では約 7.7 ms。
相対ジャンプの基準	全資料一致: オペランドの番地を基準に $\pm n$ (POPCOM は「命令の番地 $+1 \pm n$ 」と書く)。	76 か所の相対分岐すべてが命令の先頭に着地する。基準を次の命令にすると 46 か所が命令の途中になる。
TEST n(6B)	utz82: 「テストバイト」とだけ記述。 MAME・PockEmul: ビット 1 が 2 ms タイマ、ビット 3 が BRK(ON) キー。 POPCOM: 08 が BRK キー、80 がカセット入力。タイマの記述はない。	MAME・PockEmul を採用。TEST &08 で BRK キー、TEST &02 で乱数の種を作っている。
CALL の積み方	MAME・PockEmul・POPCOM: 上位を先に積み、下位が小さい番地に来る。	採用。エラー時に 58 番地 (下位) を +1 する処理が、ROM 側の RESET C(RC) を 1 バイト飛ばす動作と合う (2.9 節)。

トの意味もない。参照した実装では、MAME は $A=(PC+1)$ (READ) の長さで LOOP の動作が、PockEmul は $A=(PC+1)$ (READ) の長さが本書の解釈と異なる。本書のエミュレータは表 1.3 の「採用」の解釈で実装した。

1.6 メモリの使い方

1.6.1 アドレス空間

図 1.6 に全体の配置を示す。PC-1245 の RAM は C000～C7FF の 2 KB で、インタープリタはその中の C200～C5CF に置かれる。変数領域 C600～C69F より上は BASIC の作業領域で、インタープリタはいくつかの番地を借りて使う。

POPCOM の講座の図 4(PC-1250/1251) では、C5D0～C69F が BASIC の固定変数エリア (A～Z の 26 個 × 8 バイト = 208 バイト)、C6A0 からがシステム用のワークエリアとされている。インタープリタの本体は C5CF で終わり、ちょうど固定変数エリアの手前に収まる。インタープリタの変数・配列 (C600～C69F) と COM\$用の領域 (C5D0～) は、BASIC の固定変数エリアを別の用途に使い直していることになる。

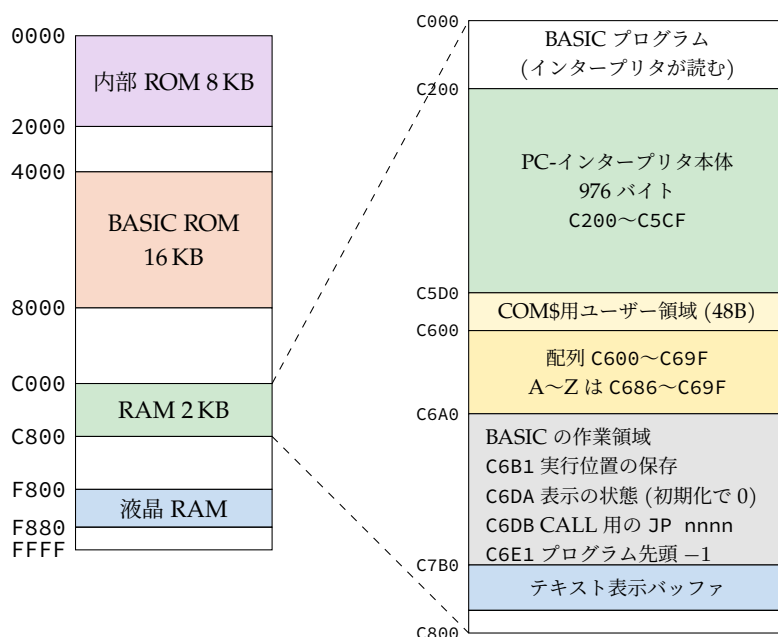


図 1.6 アドレス空間 (PC-1245)。右は RAM の拡大。BASIC の作業領域の番地は ROM の逆アドレスによる

1.6.2 CPU 内部 RAM(レジスタ)

SC61860 のレジスタは 96 バイトの内部 RAM に割り当てられている。インタープリタは A/B、I、X のほか、空いている番地を作業用に使う (図 1.7)。

■ポインタの向き {A=(+X)} (IXL) は「X を 1 進めてから読む」、{(+Y)=A} (IYS) は「Y を 1 進めてから書く」命令である。このため X は常に最後に読んだバイトを指し、ポインタの初期値は目的

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00	I	J	A	B	Xl	Xh	Yl	Yh								
10	計算 結果		BEEP DEG		エラー 番号	状態 フラグ				IF 演算子			FOR 戻り l	FOR 戻り h	FOR 回数	
20	カーソル															
30													乱数 種	乱数 種		
40																
50									戻り l	戻り h	戻り l	戻り h				ポート C

58: CALL 文への戻り (51CC) 5A: 文ディスパッチへの戻り (668A)

図 1.7 内部 RAM の使い方。黄はインタプリタの作業域、灰は BASIC/ROM が使う番地、赤は起動時のスタック

の番地の 1 つ手前に置く。ROM の 118F が X にプログラム先頭の 1 つ手前を入れるのも、11F9 が Y に C7AF を入れるのも同じ理由による。

1.7 BASIC とのつなぎ方

1.7.1 BASIC の中間コードをそのまま読む

インタプリタは行の入力、編集、トークン化を BASIC のエディタにまかせ、BASIC のプログラム領域を先頭から読む。行の形式は図 1.8 のとおりで、行番号は 2 バイト (上位は E0 に百の位を足した値、下位は下 2 桁の BCD)、行末は 00、プログラムの終わりは FF である。インタプリタは行長を使わず、行末の 00 まで 1 バイトずつ進む。

行番号 12

中間コード。キーワードは 1 バイトのトークン (RND=AE)、文字は内部コード

行末

E0	12	52	34	40	41	35	AE	40	46	1D	51	34	52	37	40	51	1D	53	34	44	40	35	52	00	
		B	=	0	1	+	RND	0	6	:	A	=	B	*	0	A	:	C	=	4	0	+	B		

文字コード: 数字 40~49、英字 51~6A、:=1D、_=1B、"=12、&=1F、)=31、>=32、<=33、>=34、+-*/=35~38

図 1.8 サンプルプログラム 12 行 B=01+RND 06:A=B*0A:C=40+B の格納形式

トークンの値は、BASIC ROM のキーワード表 (4100 台) を読んで確かめた。表の各項目は「長さ・名前・トークン・処理ルーチンの番地」の並びで、表のキーワードに対応するトークンをここで確認した。ただし、関係演算子<>はこの表に文字列としては現れない (表 1.4)。

1.7.2 ROM ルーチン

インタプリタは 2 バイト命令 CAL で内部 ROM のルーチンを 12 種類呼ぶ。PC-1251 の ROM を逆アセンブルした結果を表 1.5 に示す。PC-1245 の ROM は確認していないが、呼び出し箇所の前後のレジスタの使い方はすべてこの内容と矛盾しない。

表 1.4 インタープリタが解釈するトークン (キーワードは BASIC ROM で確認。<>は推定)

C1	PRINT	D0	IF	D7	GOTO	DB	BEEP
C9	CALL	86	NOT	D1	FOR	95	WAIT
87	SQR	92	THEN	D5	NEXT	A1	COS
AC	DEG	D4	END	D6	STOP	C5	CLEAR
89	COM\$	D2	LET	AF	PEEK	AE	RND
8A	INKEY\$	81	AND	85	OR	84	<>(推定)

表 1.5 インタープリタが呼ぶ ROM ルーチン

番地	ROM の中身 (要約)	インタープリタでの用途
118F	DP=&C6E1 (LIDP); P=&04 (LP); [(P)=(DP)] _J (MVBD): X ← (C6E1)	実行開始、GOTO の先頭からの探索
11C7	戻り番地の下に X の 2 バイトを差し込む	X の退避 (PRINT、STOP、GOTO、 INKEY\$)
11D2	上の逆。A を作業に使う	X の復帰
11AF	BASIC ROM 44FB を呼んで表示	PRINT の表示
11F9	A=B0, B=C7 で 11F5 へ	Y ← テキストバッファ -1
11F5, 11EE	Y ← BA-1, Y ← BA	DEG、SQR のポインタ設定
1899	P=&04 (LP); Q=&1C (LIQ); [(P)=(Q)] _J (MVB): X ← (1C,1D)	NEXT で戻る
1FB1	P=&5F (LIP); A ↔ (P) (EXAM); PORT(C)=(&5F) (OUTC): ポート C ← A	BEEP の音出し
11E0	A=01 で 1FB1 へ	起動時のポート C 初期化
1F44	TEST &08 で BRK なら 07、それ以外はキー スキャン	INKEY\$
1D0D	ポートと LCD を操作してキーを待つ	STOP

1.7.3 CALL で入って END で戻るまで

BASIC の CALL 文 (ROM 51AF) は、呼び出し先の番地を C6DB~C6DD に JP nnnn として書き込み、そこを CALL する。呼び出しの前後で BASIC 自身の実行位置 X を C6B1 に保存・復帰し、戻り先 51CC には 1 バイト命令 RESET C (RC) が置かれている。図 1.9 にこの経路を示す。

コラム レジスタはメモリの中にある

SC61860 の A レジスタは、内部 RAM の 02 番地そのものである。B は 03、I は 00、J は 01、X は 04・05、Y は 06・07 番地にある。「レジスタ」と呼んではいるが、実体は 96 バイトの内部 RAM の一部で、ポインタ P で番地を指して A=(P) (LDM) や A ↔ (P) (EXAM) で読み書きできる。スタックも同じ内部 RAM の 5B 番地から下へ伸びる。

P を設定する P=n (LP) は 1 バイト命令で、n は 00~3F の 64 通りをオペコードそのものに埋め込む。そのため 80~BF の 64 個のオペコードが P=l (LP)1 命令に割り当てられている。内部 ROM を呼ぶ 2 バイト命令 CAL も、番地の上位 5 ビットをオペコードに埋め込むので E0~FF の 32 個を使う。256 個のオペコードのうち 96 個、37.5% がこの 2 命令で占められている (図 1.10)。

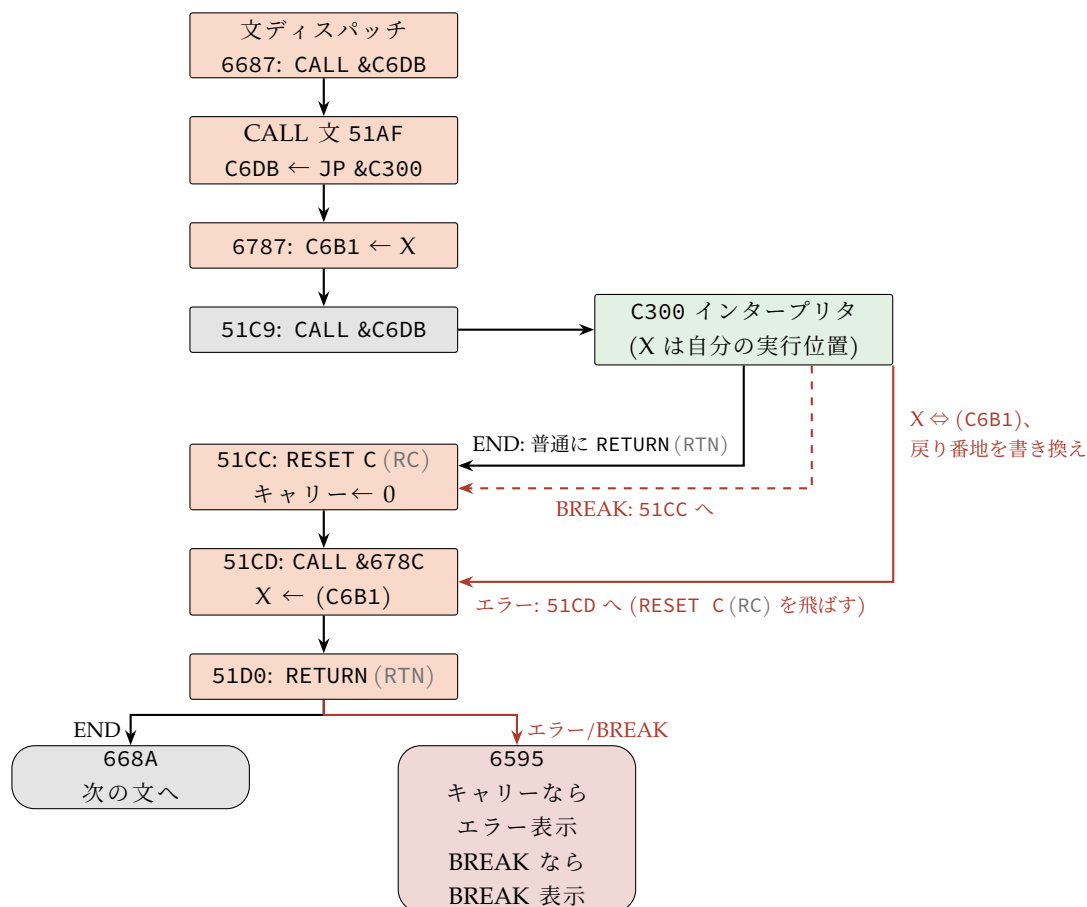


図 1.9 BASIC の CALL 文からインタープリタに入り、BASIC に戻るまで (番地は PC-1251 の ROM)。赤はエラーと BREAK の経路

レジスタをメモリに置く設計は、TI の TMS9900(16 本のレジスタを RAM 上に置き、ワークスペースポインタで場所を指す) などにも見られる。SC61860 の場合、レジスタを増やしたければ内部 RAM の番地を使えばよく、このインタープリタも 10 番地を計算結果、20 番地をカーソルと、内部 RAM を自由に「レジスタ」として使っている。

レジスタが番地であることは、命令の副作用にも表れる。POPCOM の講座の命令表は、 $A=A+1$ (INCA) や $I=I-1$ (DECI) などの増減命令の「変化レジスター」に Q を挙げている。対象のレジスタの番地 (A なら 02) が Q に残ると読める。このプログラムで Q を使うのは FOR の $[(P)=(Q)]_I$ (MVW) だけで、その直前に $Q=\&04$ (LIQ) で設定し直しているので影響はない。本書のエミュレータにもこの動作を入れたが、テストの結果は変わらなかった。

コラム 上位が先か、下位が先か

SC61860 は、命令の中の番地と、メモリに置かれた 16 ビットの値とで、バイトの並び順が逆になっている (図 1.11)。

- 命令の即値は上位が先。JP &C3AA は 79 C3 AA、DP=&C6B1 (LIDP) は 10 C6 B1 と並ぶ。

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0-	LII	LIJ	LIA	LIB	IX	DX	IY	DY	MVW	EXW	MVB	EXB	ADN	SBN	ADW	SBW
1-	LIDP	LIDL	LIP	LIQ	ADB	SBB			MVWD	EXWD	MVBD	EXBD	SRW	SLW	FILM	FILD
2-	LDP	LDQ	LDR	CLRA	IXL	DXL	IYS	DYS	JRNZP	JRNZM	JRNCP	JRNCM	JRP	JRM		LOOP
3-	STP	STQ	STR	STH	PUSH	DATA		RTN	JRZP	JRZM	JRCP	JRCM				
4-	INCI	DECI	INCA	DECA	ADM	SBM	ANMA	ORMA	INCK	DECK	INCM	DECM	INA	NOPW	WAIT	CUP
5-	INCP	DECP	STD	MVDM	MVMP	MVMD	LDPC	LDD	SWP	LDM	SL	POP		OUTA		OUTF
6-	ANIM	ORIM	TSIM	CPIM	ANIA	ORIA	TSIA	CPIA		DTC	NOPT	TEST				CDN
7-	ADIM	SBIM	LIIH		ADIA	SBIA			CALL	JP	PTC	HALT	JPNZ	JPNC	JPZ	JPC
8-	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP
9-	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP
A-	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP
B-	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP	LP
C-	INCL	DECL	INCB	DECB	ADCM	SBCM	TSMA	CPMA	INCL	DECL	INCN	DECN	INB	NOPW	NOPT	
D-	SC	RC	SR	NOPW	ANID	ORID	TSID	CPID	LEAVE	NOPW	EXAB	EXAM		OUTB		OUTC
E-	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL
F-	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL	CAL

図 1.10 SC61860 のオペコード表。上位 4 ビットを行、下位 4 ビットを列にとった。■ P=ℓ (LP)
■ CAL ■ 分岐・呼び出し ■ 機能が不明、または別の命令の別名とされるもの

- レジスタや内部 RAM の 16 ビット値は下位が先。X は 04 番地が下位、05 番地が上位である。CALL でスタックに積まれる戻り番地も、小さい番地に下位が入る。16 ビット加算の $(P+1, P) = (P+1, P) + (B, A)$ (ADB) も「(P) に下位、(P+1) に上位」として計算する。

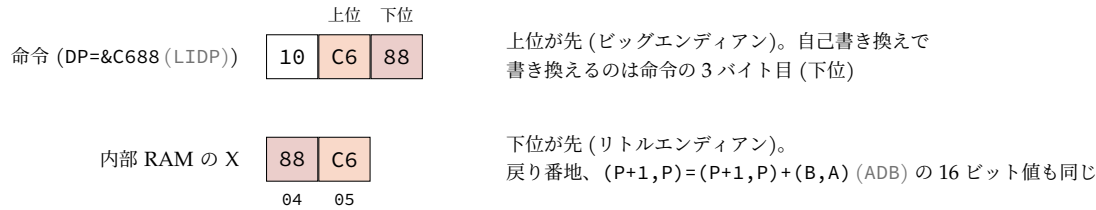


図 1.11 同じ番地 C688 の、命令の中とメモリの中での並び

1984 年の POPCOM の講座も、3 バイトの CALL について「メモリー上には 78 n m の順にならびますので、Z80 のマシン語を知っている方は、ご注意ください」と書いている。当時のパソコンで主流だった Z80 は、命令の中の番地も下位が先だったからである。

このインタープリタはこの違いをうまく使っている。自己書き換え (その 2) で書き換えるのは DP=nm (LIDP) の 3 バイト目で、ビッグエンディアンだからこそ「番地の下位」が命令の最後の 1 バイトにまとまっている。エラー時に戻り番地を書き換えるときは、リトルエンディアンなのでスタックの小さい方の番地 (58, 5A) が下位になる。

BASIC の CALL 文が作るジャンプ (1.1 節、C6DB の JP nnnn) は、命令なのでビッグエンディアンで書かれている。同じ ROM の中でも、命令を作るときとデータを扱うときで並びを使い分けているわけである。

第 2 章

しくみ

2.1 実行の骨組み

2.1.1 メインループ

図 2.1 に実行の流れを示す。各コマンドは処理を終えると C3A9 か C3AA に戻る。ここで次の 1 文字を見て、: なら次の文、00 なら次の行へ進み、それ以外ならその文字から次の文を始める。記事にある「: は一般に省ける」はこの最後の分岐による。

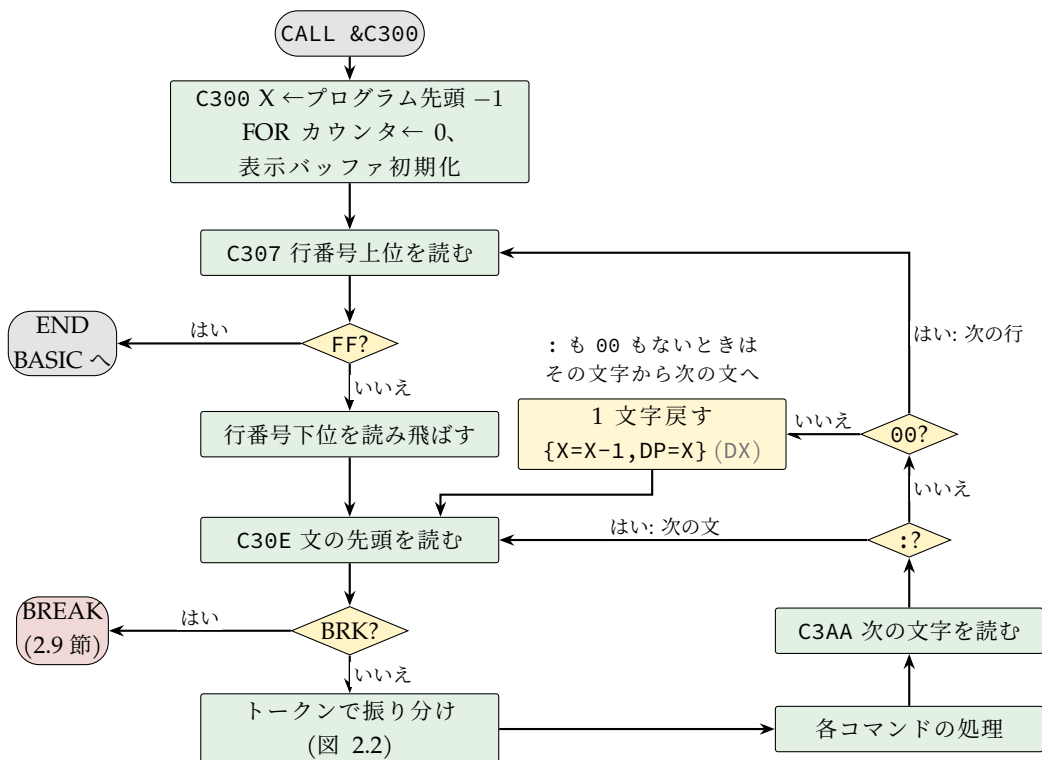


図 2.1 メインループ

2.1.2 文の振り分け

文の先頭 1 バイトを A-n (CPiA) で順に比べ、一致したところで各処理へ飛ぶ (図 2.2)。SC61860 にはテーブルジャンプ命令 (PTC/DTC) もあるが、このプログラムは比較の連鎖で振り分けている。近い飛び先は 2 バイトの相対ジャンプ、遠い飛び先は 3 バイトの JP Z,nm (JPZ) を使い分けている。変数代入は最後に判定されるので、代入文は 19 回の比較を経てから実行される。

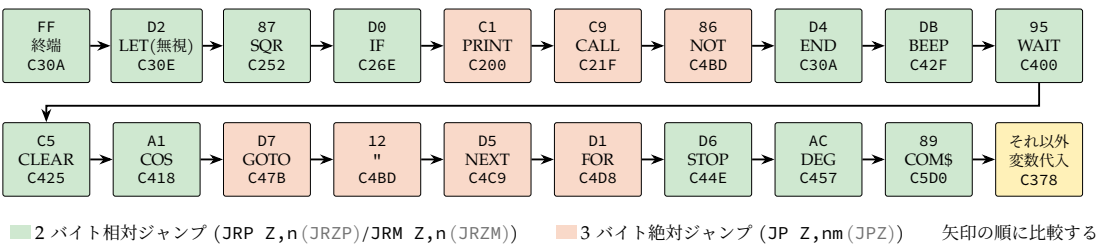


図 2.2 文の振り分けの順序。END(D4) は C30A の JP Z,nm (JPZ) に飛び込み、Z フラグが立ったまま終了処理へ進む

2.2 式の解析

2.2.1 先読み 1 文字

各ルーチンは「1 文字読んで、違っていたら {X=X-1, DP=X} (DX) で戻す」という形で書かれている。読んだ文字をもう一度読みたいときは {X=X+1, DP=X} (IX) と {A=(-X)} (DXL) を続けて使う。図 2.3 は A=B*0A の評価中の X の動きである。

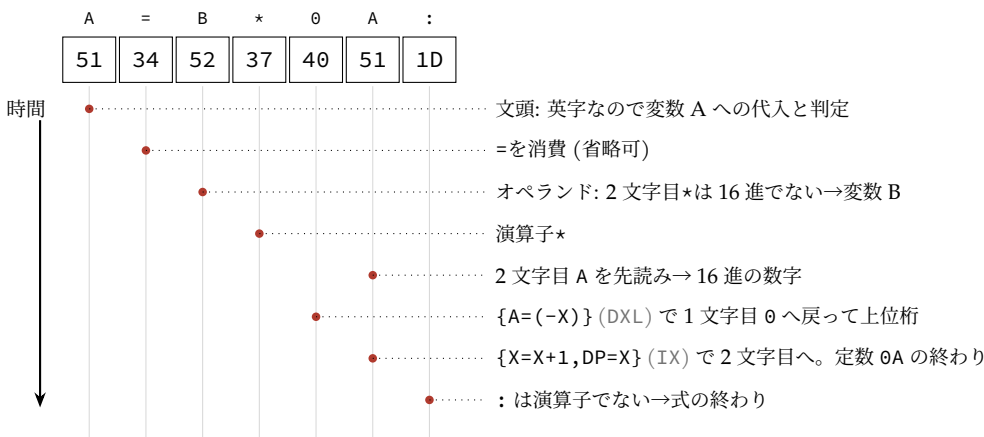


図 2.3 A=B*0A: を読むときの X の位置 (上から時間順)。X は常に最後に読んだバイトを指す

2.2.2 オペランドと演算

式は「オペランド (演算子 オペランド)…」の形で、左から順に 8 ビットで計算する。優先順位も括弧もない。結果は内部 RAM の 10 番地に置かれる。図 2.4 にオペランドの判定順を示す。

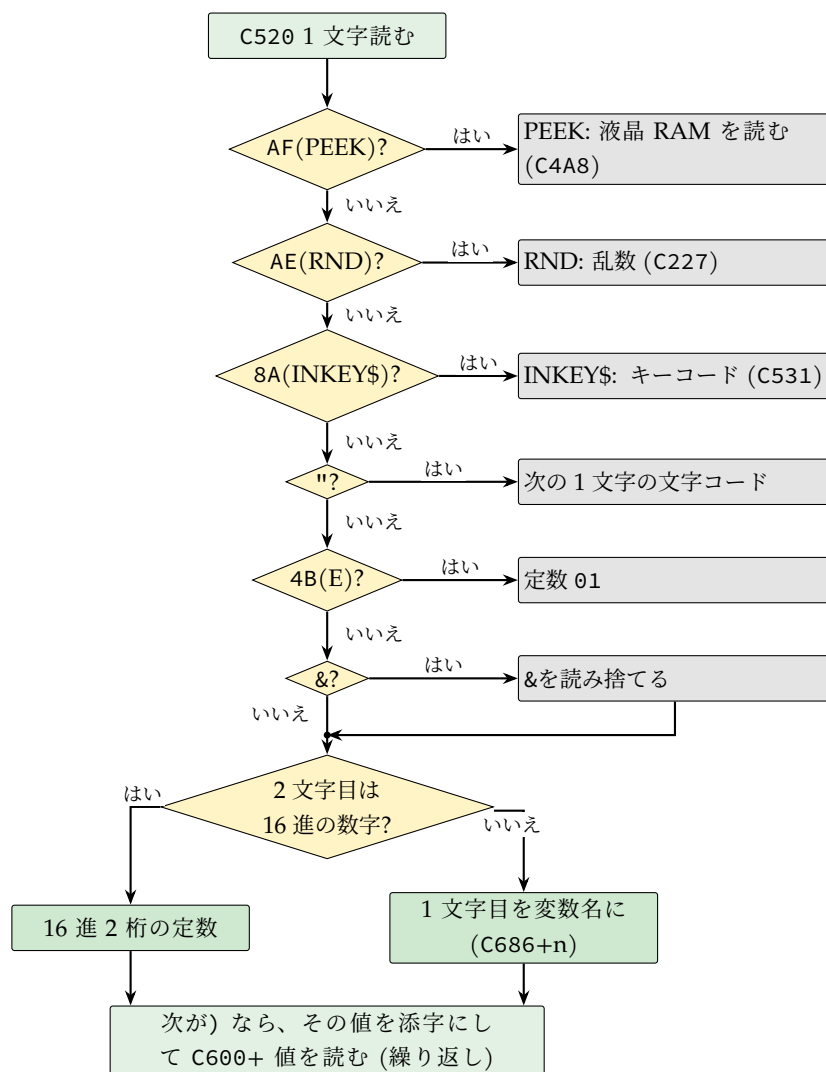


図 2.4 オペランドの判定 (C520～)

演算子 35～38(+ - * /) は C3B8、AND(81) と OR(85) は C505 で処理する。乗除算は命令がないので次のように作られている。

乗算 C3D9

2 つの値のうち小さい方を回数にして加算を繰り返す。LOOP は「回数 +1」回まわるので、最後に 1 回分を $(P) = (P) - A(SBM)$ で引いて合わせる。

除算 C3F1

引けなくなるまで引いた回数を数える。商だけが残る。除数が 0 ならエラー 2。

2.2.3 16 進定数と変数の区別

定数処理 (C5A0) は、2 文字目を先に読んで 16 進の数字かどうかを調べ、そうなら {A=(-X)} (DXL) で 1 文字目に戻って上位桁を読む (図 2.5)。2 文字目が数字でなければ、1 文字目を変数名とみなす。数字は 40~49、英字は 51 から始まるため、A~F は 40 を引いて 11~16、さらに 7 を引いて 0A~0F になる。

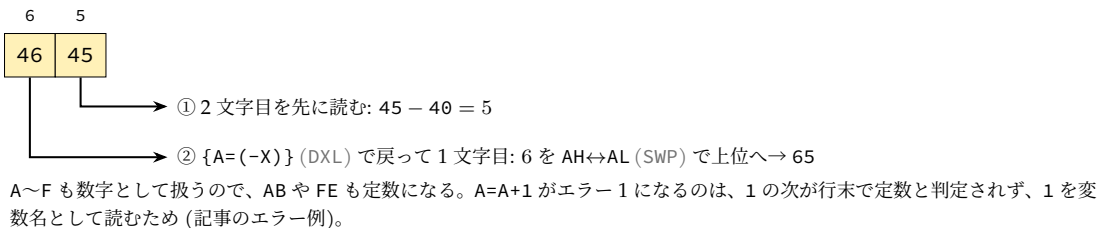


図 2.5 16 進 2 桁の読み方。下位桁を先に読んで、定数かどうかの判定を兼ねる

2.2.4 変数・配列と自己書き換え

変数 A~Z は C686~C69F の各 1 バイト、配列は C600+ 添字で、両者は同じ領域を共有する (図 2.6)。添字は値の後ろに) を付けて書く (A) なら C600+A の値)。

番地の指定には自己書き換えを使う。変数名や添字から計算した下位バイトを、すぐ後にある DP=&C6xx (LIDP) 命令のオペランドに書き込み、その命令で DP を設定する。X は実行位置で使用中、DP の下位だけを変える DPL=n (LIDL) は即値しか取れないため、この方法が短く済む。

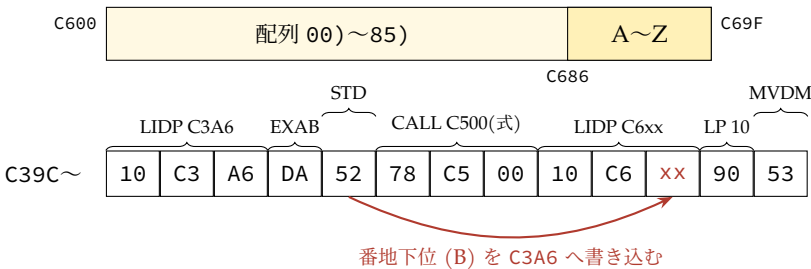


図 2.6 上: 変数と配列の領域。下: 代入 (C39C) の自己書き換え。式の結果 (10 番地) を C6xx に書く

同じ手法は変数の読み出し (C56F) と PEEK(C4B4) でも使われている。なお、配列の添字は 00 ~FF まで通るので、A0 以上を使うと BASIC の作業領域 (C6B1 や C6E1) を壊す。

2.3 略記法のしくみ

記事には「&はすべて省ける」「数値代入時の=は一般に省ける」「: は一般に省くことができる」といった注意が散らばっている。BASIC では文法エラーになる書き方が、このインタープリタでは通る。これは特別な略記の処理を書いたからではなく、パーサーの作りから自然に出てくる性質である。

2.3.1 「あれば読む、なければ戻す」

パーサーは先読み 1 文字で書かれている (1.1 節のポインタ、図 2.3)。たとえば代入の=は、次の形で読まれている。

```
{A=(+X)} (IXL)      ; 次の1文字を読む
A-&34 (CPIA)         ; '=' か?
JRP Z,OK (JRZP)      ; そうなら読んだままにする
{X=X-1,DP=X} (DX)    ; 違えば読まなかったことにする
OK:                  ...
```

期待した記号がなければ、X を 1 つ戻して何もなかったことにする。この 4 命令が置かれた記号は、書いても書かなくても同じ意味になる。表 2.1 に、この形で省略できるものと、関連する書き方をまとめた。

表 2.1 使える略記と、それを可能にしている箇所。実 ROM エミュレータで動作を確認した

書き方	しくみ	番地
A05(A=05)	変数名のあとの=を「あれば読む」	C396
A=05B=06(: なし)	文の終わりで: も行末もなければ、その文字から次の文を始める	C3AA
A=0F(A=&0F)	オペランドの前の&を「あれば読み捨てる」。定数はもともと 16 進法	C547
A=03(LET A=03)	LET のトークンは読み捨てて次の文へ	C318
IF A=01 B=02	条件の後ろの THEN を「あれば読む」	C2AF
PRINT "AB(閉じ"なし)	行末(00) でも文字列を終える	C20F
A) A)) 05)	値の後ろに) が続く限り、その値を添字にして読み直す	C578
"Z(閉じ"は不要)	"の次の 1 文字を文字コードとして読む	C53C
"で始まる文	次の NOT か行末まで読み飛ばす (注釈)	C4BD

これらを組み合わせると、図 2.7 のように 24 バイトの行が 15 バイトになる。冒頭の代入値を 05 にした場合と 06 に変えた場合の両方で、省略の有無によらず同じ結果になることを、エミュレータで確かめた。PC-1245 のプログラム領域は小さく、1 行の長さも入力の手間も減るので、略記の効果は大きい。

省略なし (24 バイト)

LET	A	=	&	0	5	:	IF	A	=	0	5	THEN	LET	B	=	A	:	NOT	LET	B	=	0	0
-----	---	---	---	---	---	---	----	---	---	---	---	------	-----	---	---	---	---	-----	-----	---	---	---	---

略記 (15 バイト)

A	0	5	IF	A	=	0	5	B	=	A	NOT	B	0	0
---	---	---	----	---	---	---	---	---	---	---	-----	---	---	---

図 2.7 同じ意味の 2 つの行。赤い 9 バイト (LET3 個、THEN、=2 個、&、:2 個) を省ける

2.3.2 A)) という書き方

配列の添字は、値の後ろに) を付けて書く。A) は「A の値を添字にした配列要素」、つまり「C600+A の値」という番地に入っている値である。変数の読み出し (C56F) は、読んだ値の後ろにまた) があれば、その値を添字にしてもう一度読む。そのため A)) は「A) の値を添字にした配列要素」になり、ポインタを 2 回たどる動作になる。A=05、A)=09、B=09、B)=77 のとき、A)) の値は 77 である (エミュレータで確認)。記事には書かれていないが、この処理がループになっていることから出てくる使い方である。

2.3.3 省略できないとき

略記は便利だが、落とし穴もある。BASIC は文字列の外の空白を保存しないとみられる (サンプルの IF INKEY\$ =C GOTO 40 が実行できるのはそのため)。すると A=B C=01 は A=BC=01 として格納され、BC が 16 進法の定数 BC と読まれてしまう。続く =01 は文の先頭として解釈できず、エラー 1 になる。変数名が A~F の文字の前に来るときは、: を省かない方が安全である。

同じ理由で、A=A+1 もエラー 1 になる (2.2.3 節)。定数はかならず 16 進法 2 桁で書く。

2.4 GOTO

GOTO は行番号の下位 1 バイトだけを比べる (図 2.8)。下位バイトは行番号の下 2 桁の BCD なので、GOTO 26 は下 2 桁が 26 の行 (26、126、426...) のうち最初の行に飛ぶ。記事の「GOTO 26 で 426 行へ」「下 2 ケタが同じなら若い行へ」はこの比較方法による。

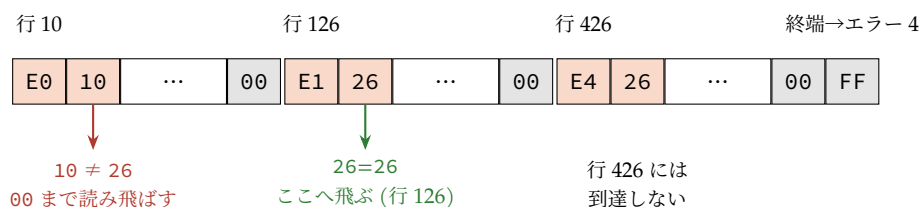


図 2.8 GOTO 26 の探索。上位バイトは FF(終端) の判定にだけ使う

探索は毎回プログラムの先頭から始め、行を 1 バイトずつ読み飛ばす。探す前に X を退避しておき (ROM 11C7)、見つからなければ戻してエラー 4 とする。見つかった場合は退避した X を POP2 回で捨てる。記事の「GOTO が遅い」「よく使うサブルーチンは先頭に置く」はこの作りによる。

2.5 IF と NOT

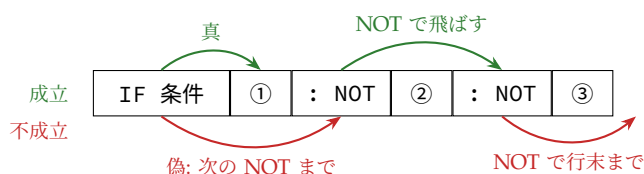
IF は左辺をスタックに積んでから右辺を評価し、比較の結果を A に FF(左>右)、00(等しい)、01(左<右) の 3 値で作る。そのあと演算子ごとに A を ± 1 ずらし、最後の A=A-1 (DECA) でポロー

(桁借り、1.1 節) が出るかどうかの 1 点で真偽を決める (表 2.2)。

表 2.2 関係演算子の判定。最後の $A=A-1$ (DECA) で判定する。<>だけはボローが出なければ真

演算子	前処理	$A=FF$ (左>右)	$A=00$ (等しい)	$A=01$ (左<右)	分岐
=	なし	×	○	×	ボローで真
<	$A=A-1$ (DECA)	×	×	○	ボローで真
>	$A=A+1$ (INCA)	○	×	×	ボローで真
<>	なし	○	×	○	ボローで偽

偽のときは、次の NOT が行末まで読み飛ばす (C4BD)。同じルーチンが NOT 文そのものと"文 (REM 相当) の処理にも使われるので、実行中に NOT に出会うと次の NOT まで飛ぶ。この 1 つのルーチンの共用だけで、ELSE に相当する動作が得られる (図 2.9)。



成立時は①と③、不成立時は②を実行する。記事の「IF ① ②: NOT ③: NOT ④: NOT ⑤で、成立時 ②→④、不成立時③→⑤」と同じ動作である。BASIC のラベル行 900: "=" CALL &C300 …も"で始まるので、インタープリタからは読み飛ばされる。

図 2.9 IF と NOT による分岐

2.6 FOR と NEXT

FOR(C4D8) は回数を 1E 番地に、現在の X を 1C・1D 番地に $[(P)=(Q)]_1$ (MVW) で保存する。NEXT(C4C9) は回数を 1 減らし、負にならなければ ROM 1899 で X を戻す。カウンタも戻り位置も 1 組しかないので、記事の P.S. のとおり 1 段しか使えない。起動時に回数を 0 にするため、FOR より先に NEXT に出会っても素通りする。コード上は FOR n で本体が $n+1$ 回実行される (エミュレータでも FOR 03 で 4 回)。

2.7 表示

2.7.1 液晶 RAM の並び

PC-1251 の液晶は 5 ドット幅の文字が 24 桁、計 120 列で (PC-1245 は 16 桁表示なので、見える範囲はこの一部と考えられる)、各列が液晶 RAM の 1 バイト (下位 7 ビットが縦 7 ドット) に対応する。左半分 60 列は F800~F83B の順、右半分 60 列は F87B から F840 へ向かう逆順に並ぶ (MAME の画面描画処理で確認)。DEG と PEEK は位置 00~77 を受け取り、3C 以上なら B7—位置で番地の下位を作る (図 2.10)。

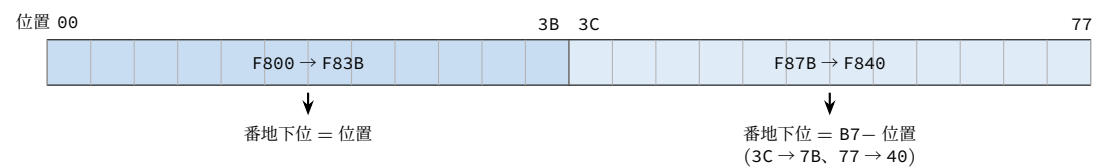


図 2.10 液晶の列と液晶 RAM の対応

2.7.2 各命令

PRINT C200

開き"の次から、閉じ"か行末までをテキスト表示バッファ (C7B0～) のカーソル位置 (20 番地) ヘコピーし、ROM 11AF で表示する。カーソルは進めず、バッファも消さない。

CALL C21F

式の値をカーソル位置にするだけ。

COS C418

液晶 RAM(120 バイト) を 0 で埋め、カーソルを 0 にしてテキストバッファを 21 で埋める。21 は ROM の表示ルーチンで空白として描かれる。

SQR C252

液晶 RAM の各バイトを FF-A で反転する。NOT 命令がないためである。

DEG C457

DEG 位置, パターン, …で、液晶 RAM に 1 列ずつ直接書く。

PEEK C4A8

DEG と同じ変換で液晶 RAM を読む。

図 2.11 は、実 ROM を載せたエミュレータで描いた画面である (液晶 RAM の内容をそのまま図にした)。



(a) サンプル 10～14 行: PRINT "= 1 2 3 4 5 6 = PC-1251"のあと DEG A,40,60,60,60,40(A=28)



(b) CALL 01:PRINT "GAME OVER ":SQR



(c) DEG 00,7F,41,41,41,7F:DEG 3B,7F:DEG 3C,7F:DEG 77,7F

図 2.11 エミュレータ (実 ROM 使用) で得た液晶の内容

2.8 時間と音

2.8.1 WAIT

WAIT(C400) は内側に WAIT &C8、WAIT &FF、WAIT &FF を置き、BRK キーを見ながら LOOP で回す。継続する 1 周は 748 サイクル (utz82 の表) で、クロックを 192 kHz とすると約 3.90 ms、ほぼ 1/256 秒になる。記事の「 $n/256$ 秒の時間待ち」と合う。LOOP の性質上、 $n \geq 1$ では $n+1$ 周、 $n=0$ では待たない。

2.8.2 BEEP

BEEP(C42F) は、ROM 1FB1 を通してポート C に 51 と 41 を交互に書く。2 つの値はビット 4 だけが違い、圧電ブザーを直接振動させる。半周期の遅延には、書き込めない ROM 番地 0000 への [(DP)=A] (FILD) を使う。[(DP)=A] (FILD) は $4+3I$ サイクルかかり、ROM なので内容は変わらない。つまり「時間だけがかかる命令」として使っている (図 2.12)。

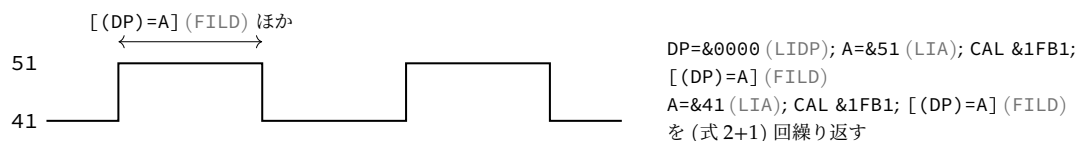


図 2.12 BEEP の波形 (模式図。高・低の時間は異なる)。ROM への [(DP)=A] (FILD) が遅延になる

■ポート C の中身 ROM 1FB1 は $P=\&5F$ (LIP)、 $A \leftrightarrow (P)$ (EXAM)、 $PORT(C)=(\&5F)$ (OUTC) の 3 命令で、A を内部 RAM の 5F 番地 (ポート C の出力バッファ) に置いてから出力する。ポート C の各ビットの意味を、POPCOM の講座の表 7 と MAME のソースの注釈 (PC-1401 系の説明) で読み合わせると、表 2.3 のようになる。2 つの資料の読みは一致している。

BASIC の BEEP 文 (ROM 5140。キーワード表で処理番地を確かめた) は 30 を書き、内蔵の発振回路に 4 kHz の音を出させる。音の高さは固定で、表示のビット 0 は 0 になっている (資料どおりなら、鳴っている間は表示が消える)。インタプリタはビット 5 を 0 にして発振を止め、ビット 4 を自分で反転させて振動板を動かす。これで高さを自由に変えられ、ビット 0 が 1 のままなので表示も残る。

エミュレータでサイクル数を数えると、BEEP 03 で約 2.1 kHz、BEEP 09 で約 1.5 kHz、BEEP 20 で約 0.7 kHz になる。式 1 が大きいほど音は低い。記事の「式の値が大きいと高い (長い) 音」とは逆だが、値はサイクル表とクロックの仮定に依存する。

2.8.3 RND

RND(C227) は、 $n-1$ を覆う最小の $2^k - 1$ のマスクをシフトで作り、乱数にマスクをかけて $n-1$ を超えたら引き直す (棄却法)。元の乱数が一様なら、剰余による範囲の変換で生じる偏りを避

表 2.3 ポート C のビット (POPCOM の講座の表 7 と MAME の注釈による) と、書き込む値

ビット	POPCOM の講座	MAME の注釈	起動時	BASIC の BEEP	インタープリタ
			01	30	51/41
0	表示のオン/オフ	display on	1	0	1
1	カウンタのリセット	counter reset	0	0	0
2	クロック停止 (HALT)	cpu halt	0	0	0
3	電源オフ	computer off	0	0	0
4	カセットとブザー (注)	4 kHz(1)/2 kHz(0)。発振を止めているときは振動板の位置	0	1	1/0
5	カセットとブザー (注)	beeper on(発振する)	0	1	0
6	カセットとブザー (注)	beeper control	0	0	1

(注) 講座はビット 6~4 をまとめて説明し、011 なら 4 kHz、010 なら 2 kHz を出し、101 と 100 で出力を HIGH と LOW にしている。

けられる。乱数は内部 RAM3C・3D 番地の値と、TEST &02(2 ms ごとに反転するタイマ) が変わるまで数えたカウンタを混ぜて作る。

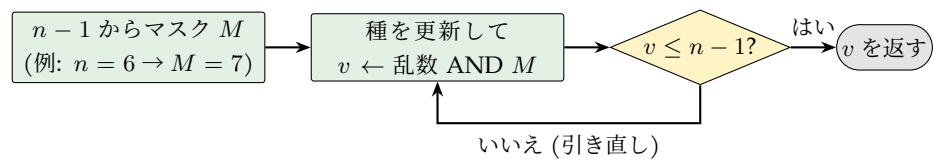


図 2.13 RND の手順

2.9 エラーと BREAK

エラー (C584) と BREAK(C58F) は、BASIC が自分のエラーとして処理できるように、戻る前に 2 つの細工をする (C2E4)。

1. X と C6B1 の内容を交換する。BASIC の CALL 文は戻ったあと C6B1 から自分の X を戻すので、BASIC は「インタープリタが実行していた位置」を受け取り、その行番号でエラーや BREAK を表示する。記事の「エラー番号と行番号が出て、↑でその部分にカーソルが点灯」はこの仕掛けによる。
2. 内部 RAM5A・5B 番地にある文ディスパッチへの戻り番地から、下位 F5・上位 01 を桁借りなしで引く。668A が 6595(もう 1 つのディスパッチ箇所への戻り番地) に変わり、BASIC は CALL の次の文へ進まずに止まる (図 2.14)。状態フラグ (15 番地) が 10 のときは書き換えない。

エラー時はさらに 58 番地の戻り番地の下位を 1 増やし、キャリーを立てて戻る。戻り先が 51CC の RESET C (RC) を飛ばして 51CD になるので、キャリーが残り、BASIC はエラー (番号は 14 番地) として扱う。BREAK では状態フラグを 30 にして、キャリーを立てずに戻る。

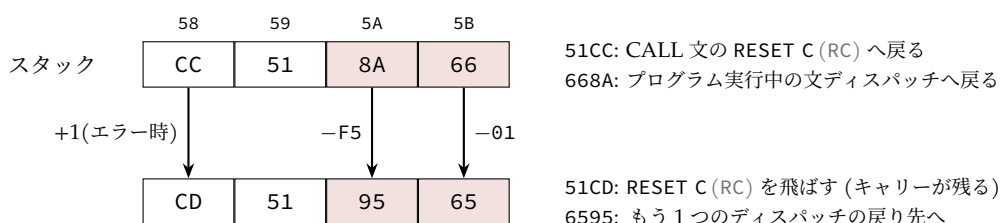


図 2.14 エラー時の戻り番地の書き換え (番地は PC-1251 の ROM)

コラム 戻り番地と戻り値

SC61860 のサブルーチンには、値を返すための決まった形がない。このインタープリタと ROM のルーチンは、場面ごとに次のような手段を使い分けている。

返し方	例	使われ方
A レジスタ	C372(EVAL_A)、ROM 1F44	式の値、キーコード
内部 RAM の番地	10 番地 (式の値)、14 番地 (エラー番号)	呼び出しをまたいで残す値
キャリーフラグ	インタープリタ→BASIC	エラーかどうか
Z フラグ	C335 の JRM Z,n(JRZM)	直前の比較結果をそのまま使う
戻り番地そのもの	エラー時の 51CC → 51CD	戻る場所を変えて結果を伝える

最後の「戻り番地を変える」は、今の CPU の呼び出し規約から見ると奇妙に映る。CALL 文の戻り先 51CC には 1 バイトの RESET C(RC) が置かれていて、普通に返るとキャリーが消える。インタープリタはエラーのとき戻り番地の下位を 1 増やし、この RESET C(RC) を飛ばしてキャリーを立ててたまま戻る。1 つの呼び出しに「成功用」と「失敗用」の 2 つの戻り口を用意していることになる。

文ディスパッチの戻り番地 668A を 6595 に変える処理も、1 バイトずつの $(P) = (P) - n$ (SBIM) で下位から F5、上位から 01 を引いている。 $(P) = (P) - n$ (SBIM) は桁借りを上位に伝えないが、下位の 8A から F5 を引けば必ず桁借りが出るので、その分を上位から 1 引いておけばよい。結果は $668A - F5 = 6595$ で、16 ビットの引き算をした場合と同じになる。戻り番地が決まっているからこそ使える書き方である。

コラム 前向きと後ろ向きで別の命令

多くの CPU の相対ジャンプは、符号付きのオフセット (-128~+127) を 1 つの命令で扱う。SC61860 は向きごとに命令を分けている。JRP と JRM、JRP Z,n(JRZP) と JRM Z,n(JRZM) のように、P は前 (plus)、M は後ろ (minus) で、オフセットは符号なしの 0~255 である。前後それぞれ 255 バイトまで届く代わりに、条件ごとに 2 つずつオペコードが要る。基準はオフセットのバイト自身の番地で、このプログラムの 76 か所の相対分岐はすべてこの規則で命令の先頭に着地する。

POPCOM の講座の表 1 は、命令の形を 8 ビットのオペコードのほかに「2 ビットオペコード + 6 ビットオペランド」「7 ビットオペコード + 9 ビットオペランド」「3 ビットオペコード + 13 ビットオペランド」に分けている。2+6 は P=l(LP)、3+13 は CAL である。7+9 について講座は、マニュアルに説明がない形だとしている。本書の見方では、これは相対ジャンプにあたる。向きだけが違う組は 2C と 2D、38 と 39 のように最下位ビットだけが異なるので、このビットを符号とみなせば「7 ビットのオペコード + 符号 1 ビットと距離 8 ビット」の命令と読める。P と M に分かれた命令は、符号と絶対値で表した 9 ビットのオフセットを持つ 1 つの命令と見ることできる。

2 バイトの CAL も珍しい。Z80 には 1 バイトで 8 か所の固定番地を呼ぶ RST があるが、SC61860 の CAL は

2 バイトで内部 ROM の 0000~1FFF のどこでも呼べる。番地はオペコードの下位 5 ビットと次の 1 バイトを合わせた 13 ビットで、 2^{13} バイト=8 KB は、CPU に内蔵された ROM の大きさとちょうど同じである (講座の図 1 と本文)。このインタープリタが呼ぶ 12 種類の ROM ルーチンはすべてこの範囲にあり、3 バイトの CALL を使わずに済んでいる。

第 3 章

ここが凄い!

ここからは、976 バイトに詰め込まれた工夫を 1 つずつ取り上げる。どれも、SC61860 の癖と BASIC ROM の作りをよく調べていなければ出てこない書き方である。サイクル数は utz82 の表による概算、実測とあるのは実 ROM を載せたエミュレータで数えた値である。

ここが凄い! その 1 BASIC に「間借り」する

全体

普通にインタプリタを作るなら、行の入力と編集、キーワードのトークン化、行番号順の格納、エラーの表示まで自分で書かなければならない。PC-インタプリタはこれを 1 バイトも書いていない。プログラムは BASIC のエディタでそのまま入力し、BASIC がトークン化して保存した中間コードを先頭から読むだけである。

変えているのはキーワードの「意味」だけだ。COS は画面消去、SQR は画面反転、DEG はドット描画になる。BASIC のエディタがトークン化できるキーワードを選び、その意味を付け替えている。エラーの表示まで BASIC にやらせている (その 12)。

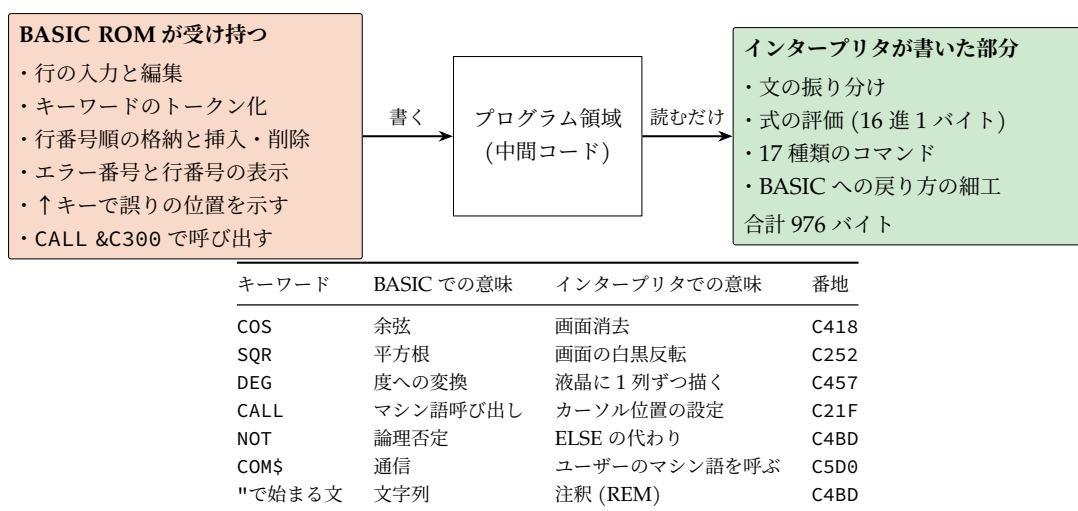


図 3.1 BASIC とインタプリタの分担。キーワードは意味だけを付け替えて使う

副産物もある。BASIC のプログラムの末尾に置いた `900:"=" CALL &C300` …のようなラベル行は、先頭が"なのでインタプリタからは注釈として読み飛ばされる。起動用の BASIC 行とインタプリタ用の行が同じプログラムに同居できる。

ここが凄い! その2 命令のオペランドを書き換えて番地を作る

C38A C39C C4B4 C56F

変数 C を読むには C688 番地を読めばよい。ところが SC61860 には「レジスタの値を番地にしてメモリを読む」命令が X、Y、DP を経由するものしかない。X は実行位置で使用中で、DP の下位だけを変える `DPL=n (LIDL)` は即値しか取れない。

そこで、計算した番地の下位バイトを、すぐ後にある `DP=&C6xx (LIDP)` 命令の 3 バイト目に書き込んでしまう。次の瞬間にその命令が実行され、DP が目的の番地になる (図 3.2)。

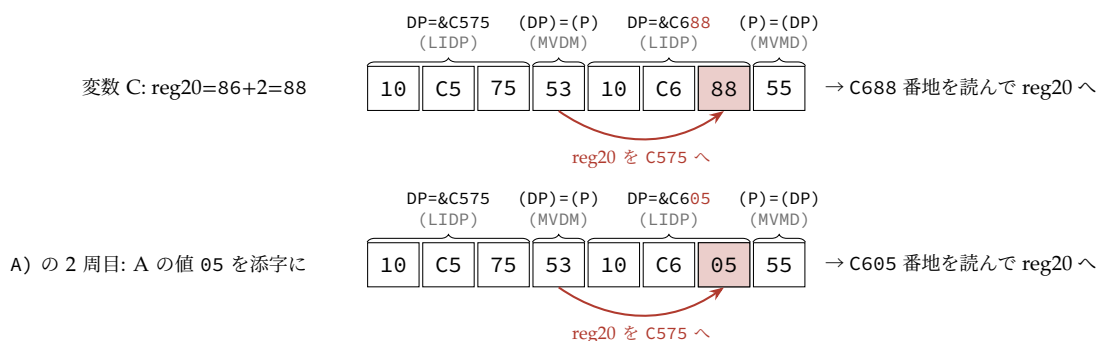


図 3.2 変数読み出し (C56F)。C575(色付き) が実行のたびに書き換わる。) が続けば同じ 4 命令をもう一度回る

Y レジスタを使って同じことをすると、`A=(P) (LDM)`、`B=&C6 (LIB)`、`CAL &11F5`、`{Y=Y+1, DP=Y} (IY)`、`P=&20 (LP)`、`(P)=(DP) (MVMD)` の 6 命令で 8 バイト、ROM ルーチンの呼び出しを含めて 65 サイクルかかり、Y と B も壊れる。自己書き換えなら 8 バイト、22 サイクルで、Y や B の値も保てる。同じ手は代入、PEEK でも使われている。

ここが凄い! その3 次の命令の 1 バイトを定数として読む

C41B C428

画面消去 (COS) と変数クリア (CLEAR) は、どちらも「A を 0 にして `[(DP)=A] (FILD)` で埋める」処理である。A を 0 にするには普通 2 バイトの `A=&00 (LIA)` を書く。作者はオペコード 56 の未公開命令 `A=(PC+1) (READ)` を使った。この命令は次の番地の 1 バイトを A に読むが、そのバイトを飛ばさない。すぐ後に置いた `I=&3B (LII)` のオペコードが 00 なので、それがそのまま A に入る (図 3.3)。

節約できるのは 1 バイトずつだが、この 1 バイトのために資料の少ない未公開命令を調べて使っている。1.5 節で見たとおり、この命令を 2 バイト命令とする資料もあり、その解釈ではここが働かない。記事のプログラムが実機で動いていたなら、それ自体が `A=(PC+1) (READ)` は 1 バイト命令だという証拠になる。

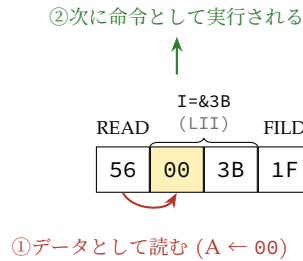


図 3.3 1 バイトの 00 が、データと命令の 2 役をこなす

ここが凄い! その 4 書けない ROM に書いて時間を稼ぐ

C43C

BEEP は、ポート C のビット 4 を 51 と 41 で交互に切り替えて圧電ブザーを鳴らす。音の高さは切り替えの間隔で決まるので、式 1 に比例する待ち時間が必要。

ここでは $DP = \&0000$ (LIDP) のあと $[(DP) = A]$ (FILD) を実行する。 $[(DP) = A]$ (FILD) は「A の値を DP 番地から $I + 1$ バイト書く」命令で、 $4 + 3I$ サイクルかかる。書き込み先の 0000 番地は内部 ROM なので、何も変わらない。残るのは $I (= \text{式 1})$ に比例した時間だけである。専用の遅延ループを書かず、1 バイトの命令で可変長の待ちを作っている。

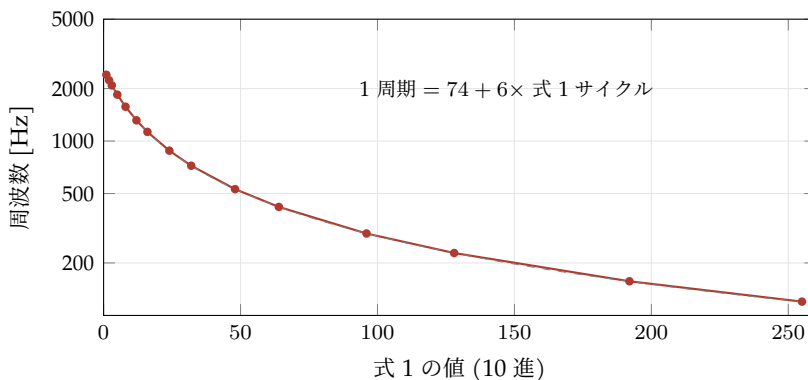


図 3.4 BEEP の周波数 (実測、192 kHz 換算)。点は測定値、破線は式。式 1 が大きいほど低い

ループの先頭で毎回 $DP = \&0000$ (LIDP) をやり直しているのにも意味がある。 $[(DP) = A]$ (FILD) は DP を I だけ進めるので、やり直さなければ DP は周期ごとに進み、いずれ C000 番地からの RAM、つまりプログラムを塗りつぶす。

ここが凄い! その 5 よその JP を借り、フラグごと飛び込む

C26C C335

遠くへ飛ぶ JP は 3 バイト、近くへの相対ジャンプ JR は 2 バイトである。作者は、近くにある別のルーチンの JP 命令へ JR で飛び、その JP に遠くまで運んでもらう (図 3.5)。

(b) が面白い。C30A の JP Z, nm (JPZ) は、本来は行番号上位が FF (プログラム終端) かどうか

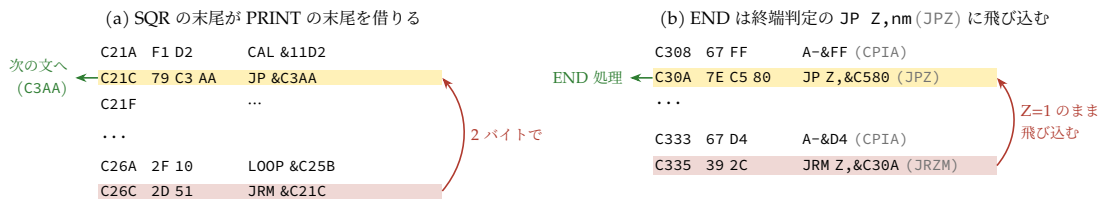


図 3.5 JR(2 バイト) で近くの JP(3 バイト) へ飛ぶ。(b) では直前の比較で立った Z フラグが、飛び込み先の JP Z,nm (JPZ) でそのまま使われる

を判定するためのものである。END(D4) と一致した直後は Z=1 なので、そこへ飛び込めば無条件に終了処理へ進む。同じ理由で FF(C314) も C30A へ飛ぶ。

ここが凄い! その 6 比較結果を 3 値にして、ポロー 1 本で判定する

C27B~C2AA

IF の関係演算子は=、<、>、<>の 4 つある。作者はまず比較結果を A に FF(左>右)、00(等しい)、01(左<右) の 3 値で作る。そのあと演算子ごとに A を ±1 ずらし、最後の A=A-1 (DECA) で「00 から 1 を引いてポロー (桁借り) が出たか」だけを見る (図 3.6)。判定の出口は 1 か所で済む。

	左>右	左=右	左<右
比較結果	FF	00	01
= なし	FF	00	01
< A=A-1 (DECA)	FE	FF	00
> A=A+1 (INCA)	00	01	02
<> なし	FF	00	01

最後に A=A-1 (DECA) を実行し、緑の 00 だけがポローを出す。
=、<、>はポローで真、<>だけはポローなしで真とする。

図 3.6 演算子ごとの前処理と、A=A-1 (DECA) 直前の A の値。ポローが出るのは 00 の枠だけ

<の場合は A=A-1 (DECA) を 2 回、>の場合は A=A+1 (INCA) のあと A=A-1 (DECA) と、短い命令の組み合わせだけで 4 つの演算子を処理している。

ここが凄い! その 7 読み飛ばし 1 つで ELSE を作る

C4BD

IF が偽のとき、NOT 文に出会ったとき、"で始まる文に出会ったとき、この 3 つをすべて同じルーチン (次の NOT か行末まで読み飛ばす) で処理している。その結果、インタープリタは図 3.7 の 2 つの状態を行き来することになり、IF ... : NOT ... : NOT ... で ELSE に相当する分岐が書ける。

ELSE 用のフラグも、入れ子を管理するスタックも持たない。記事の「成立時は②→④、不成立時は③→⑤」という少し不思議な説明は、この 2 状態のスイッチとして読むとすっきり理解できる。

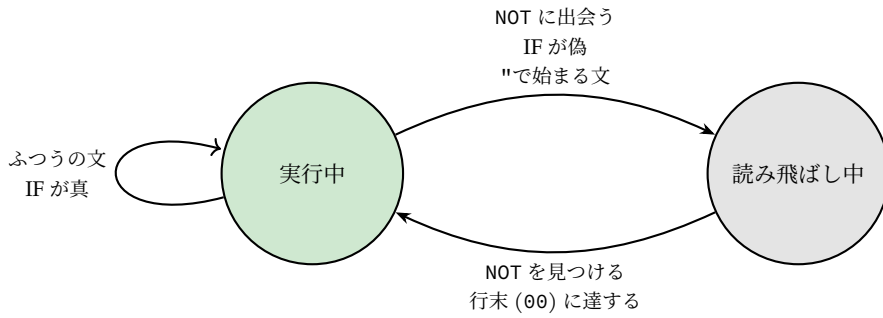


図 3.7 NOT は「状態を反転するスイッチ」として働く

ここが凄い! その 8 16 進は 2 文字目から読む

C5A0

AB は 16 進の定数なのか、変数 A の後ろに B が続いたものなのか。1 文字目を見ただけでは決まらない。作者は先に 2 文字目を読み、それが 16 進の数字なら定数、そうでなければ 1 文字目を変数名とする。先読み 1 回で区別がつく (図 2.5)。

文字コードから値への変換も短い。数字は 40~49、英字は 51 からなので、40 を引いてから、10 以上なら 7 を引く (図 3.8)。

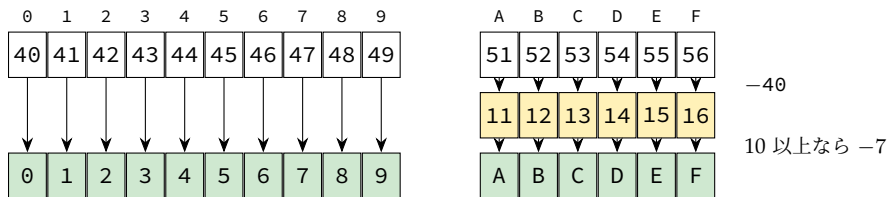


図 3.8 文字コードから 16 進の値へ。A=A-&40 (SBIA)、A-&0A (CPIA)、A=A-&07 (SBIA) の 3 命令

ここが凄い! その 9 偏らない乱数

C227

0~n-1 の乱数を作るとき、8 ビットの乱数を n で割った余りを使うと偏りが出る。たとえば n = C0(192) なら、0~63 は 256 通りのうち 2 通りずつ、64~191 は 1 通りずつしか現れない。

RND は割り算をしない。n-1 を覆う最小の 2^k-1 のマスク (C0 なら FF) をシフト命令で作り、乱数にかける。結果が n-1 を超えたら引き直す。元の乱数が一様なら、こうすると 0~n-1 が等しい確率で出る (図 3.9)。最初の試行を含む試行回数は平均 2^k/n 回で、C0 なら 1.33 回である。エミュレータでも 1 回目で決まる割合は 75.3% だった (理論値 75%)。

乱数の種には、内部 RAM3C・3D 番地の値に加えて、TEST &02 で見える 2ms タイマが切り替わるまでの待ち回数を混ぜている。いつ RND を呼んだかが結果に影響する。なお、エミュレータのタイマは乱数で代用しているため、図の結果はマスクと引き直しの部分の確認にとどまる。

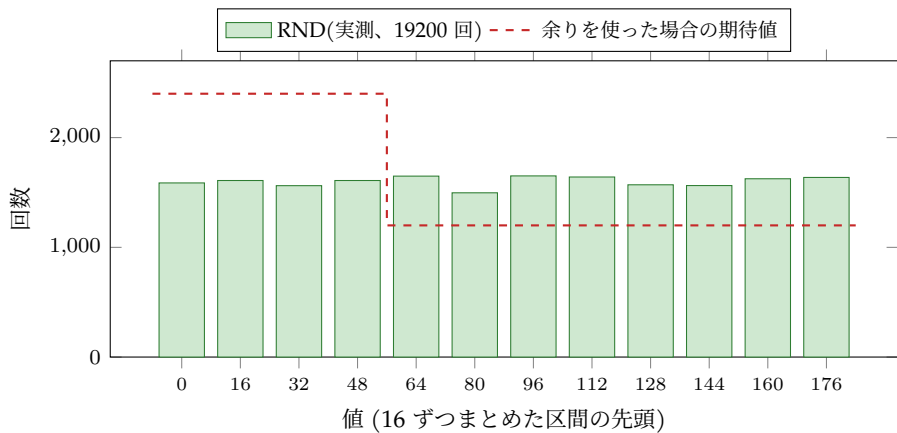


図 3.9 RND C0 の分布。棄却法では各区間がほぼ 1600 回で揃う

ここが凄い! その 10 1/256 秒に合わせた WAIT

C408

記事は「WAIT n で n/256 秒待つ」と説明している。内側のループを数えると、WAIT &C8、WAIT &FF、WAIT &FF に、BRK キーの確認とループ命令を足して 748 サイクルになる (図 3.10)。CPU クロック 192 kHz で 3.90 ms、1/256 秒は 3.91 ms である。WAIT の値を C8 にして帳尻を合わせたと見られる。

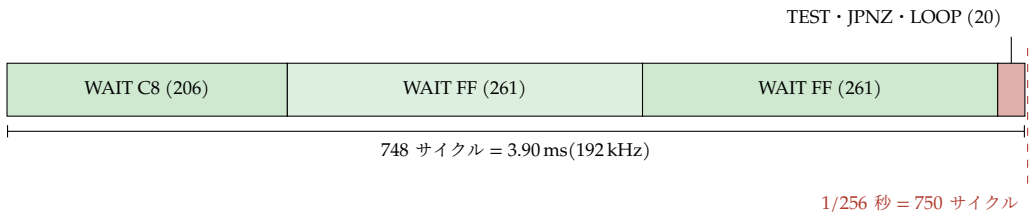


図 3.10 WAIT の内側ループ 1 周の内訳 (utz82 のサイクル表による)

ここが凄い! その 11 掛け算は小さい方を回数にする

C3D9

掛け算は足し算の繰り返しだが、作者は 2 つの値を比べ、小さい方を繰り返しの回数にしている。FF*02 なら、255 回ではなく数回の足し算で済む。さらに LOOP 命令が「回数 +1」回まわる性質を逆手に取り、最後に 1 回分を (P)=(P)-A (SBM) で引いて答えを合わせる。この方法がどのくらい速いかは 4.2 節で測った。

ここが凄い! その 12 BASIC のエラー表示を乗っ取る

C2E4 C584

インタープリタの中でエラーが起きたとき、自前でエラーメッセージを出す代わりに、BASIC に

「いま BASIC のこの行でエラーが起きた」と思い込ませる。仕掛けは 2 つある (図 1.9、図 2.14)。

1. BASIC は CALL の前に自分の実行位置を C6B1 に保存し、戻ってきたら読み戻す。インタープリタはこの保存場所と自分の実行位置 X を交換してから戻る。BASIC はインタープリタが読んでいた行を「自分の位置」として受け取るので、その行の番号でエラーを表示し、↑キーで誤りの位置にカーソルを出す。
2. CALL 文の戻り先 51CC には、キャリーを 0 にする RESET C (RC) が置かれている。エラー時は戻り番地の下位を 1 増やしてこの RESET C (RC) を飛ばし、キャリーを立てたまま戻る。BASIC はこれを自分のエラーとして扱う。

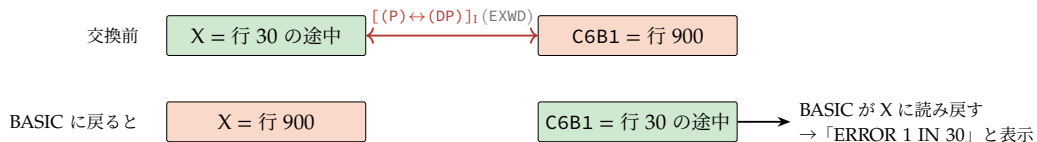


図 3.11 実行位置の交換。BASIC は行 30 で自分がエラーを起こしたと受け取る

これで、エラー番号も行番号も↑キーによる位置表示も、BASIC と同じ見た目が出る。BREAK キーで止めたときの「BREAK IN 行番号」も同じ仕掛けで表示される。インタープリタ側に表示のコードは 1 バイトもない。

コラム 割り込みのない CPU

SC61860 には割り込みがない。キーボードもタイマも、プログラムが自分で見に行く (ポーリングする) しかない。タイマは TEST 命令で読めるテストビットとして用意されていて、ビット 0 が 512 ms、ビット 1 が 2 ms ごとに切り替わる (MAME と PockEmul の実装による)。BRK(ON) キーもビット 3 で見える。

このインタープリタが文を 1 つ読むたびに TEST &08 を実行しているのは、割り込みで BRK キーを受け取れないからである。RND(その 9) が 2 ms タイマの切り替わりを待つ回数を乱数の種にしているのも、このテストビットを使っている。

待ち時間を作る専用命令 WAIT n(6 + n サイクル) があるのも特徴である。今の CPU なら割り込みやタイマで済ませる処理を、命令で時間をつぶして作る。utz82 の資料によれば、HALT 命令は PC-1245 では止まらずにメモリ全体を読み続けるテスト動作に入り、消費電力が大きく増えるという。

コラム 非公開だった命令表

SC61860 の命令表は、長いあいだシャープが一般に公開したものではなかった。oldsce の作者の小笠原博之氏は、当時のユーザーが自力で命令を解析して雑誌に発表し、のちに海外向けの資料をもとにした解説書が出たと振り返っている。命令の名前 (ニーモニック) が資料ごとに違うのはこの名残で、同じ 56 が utz82 では LDPC、MAME や oldsce では READ と呼ばれている。

1984 年の POPCOM の講座はその早い例である。編集部は米国で販売されていたシャープの英文マニュアルを、ロサンゼルス知人を通じて入手した。講座は、シャープが公表したものではないとして、読者にシャープへの問い合わせを控えるよう求めている。マニュアルにも説明のない命令があり、講座は 4 バイト以上の命令 (テーブルジャンプの PTC など) を「今後の研究課題」とし、56 も命令表の空欄のままである。一方で、ブロック交換 $[(P) \leftrightarrow (DP)]_1 (EXWD)$ は内部 RAM の 10~37、4F~5B を相手にすると正しく動かないようだ、とい

う観察も載せている。このプログラムが EXWD を使うのは X との交換 (P=04) の 1 か所だけで、その範囲には入らない。

いまでも細部ははっきりしない。utz82 の表は「機能が不明な命令」として 16、17、2E など 16 個を挙げている (図 1.10 の灰色のうち 16 個。残り 4 個は別の命令の別名とされる)。1.5 節で見たとおり、READ の長さや LOOP の終わり方は、今も主なエミュレータの間で実装が分かれている。

1986 年の記事のプログラムは、その READ と LOOP を当たり前のように使っている。作者が実機で確かめながら書いたとすれば、このプログラム自体が、未公開命令の動作を伝える資料の 1 つになっている。

第 4 章

別の書き方と比べる

この章では、同じ処理を別の方法で書いた場合と比べる。実装できたものはエミュレータで動作と速さを測り、実装していないものは見積もりとして示す。

4.1 振り分けの順序

文の振り分け (C30E) は、トークンを 19 回比べたあとで初めて「変数代入」と判定する。一方、プログラムでいちばん多く現れる文はおそらく代入である。各処理ルーチンに着くまでのサイクル数を実測すると図 4.1 のようになり、代入は 186 サイクルと最も遅い。

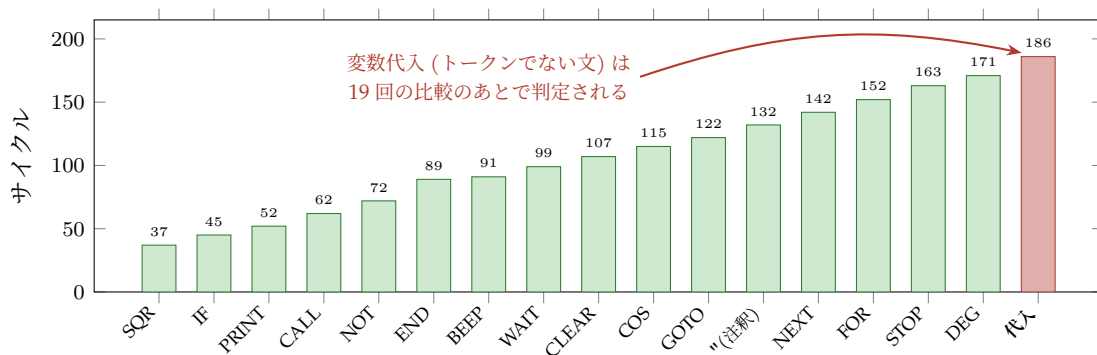


図 4.1 文の先頭を読んでから処理ルーチンに着くまでのサイクル数 (実測)。右端の赤が変数代入

トークンは 80 以上、英字は 51~6A なので、振り分けの最初に A-&6B (CPiA) と JRP NC,n (JRNCP) でトークンかどうかを分ければ、代入は約 20 サイクルで判定できる。A=01 の 1 文は実測 617 サイクルなので、所要時間を約 3 割減らせる計算である。代わりに他の文は 10 サイクル強遅くなる。バイト数はほとんど変わらない。

4.2 シフトと加算による掛け算

8 ビットの掛け算は、シフトと足し算を 8 回繰り返せば値の大きさによる時間の変動を小さくできる。次のコードを書き、65536 通りすべての組み合わせで正しいことをエミュレータで確かめた (18 バイト。現行は 13 バイト)。

```

A↔B (EXAB)          ; B=y
A=(P) (LDM)          ; A=x      (P=10)
(P)=(P).AND.&00 (ANIM) ; (10)=0
I=&07 (LII)           ; 8回
ML: A↔B (EXAB)        ; A=y, B=x
    RESET C (RC)
    SHIFT R (SR)      ; y>>=1, c=最下位ビット
    A↔B (EXAB)        ; A=x, B=y
    JRP NC,+2 (JRNCPL) ; ビットが0なら足さない
    (P)=(P)+A (ADM)    ; (10)+=x
    RESET C (RC)
    SHIFT L (SL)      ; x<<=1
    I=I-1 (DECI)
    JRM NC,ML (JRNCML)

```

結果は図 4.2 のとおりで、現行の方法は小さい方の値が 17(11) 以下なら速い (図は片方を FF に固定)。このインタープリタで掛け算に使う値はふつう小さい (サンプルでも $B \times 0A$) ので、現行の方法の方が有利な場面が多い。

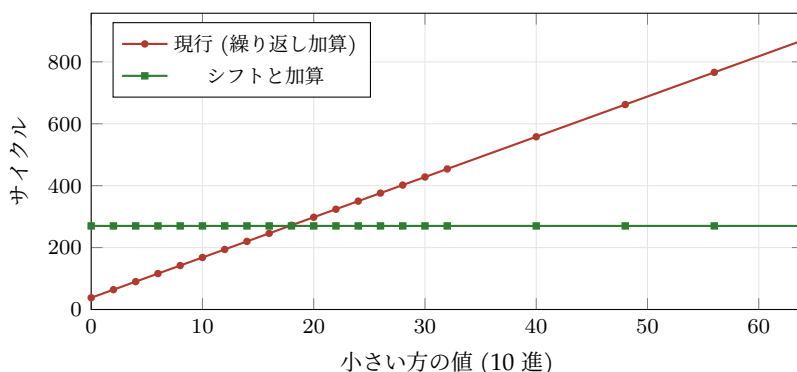


図 4.2 掛け算の所要サイクル (実測)。FF*FF では現行 3353、シフト 270

割り算は事情が違う。引けなくなるまで引く方法なので、商が大きいほど遅い。A=FE/01 の 1 文は実測 4555 サイクル (約 24 ms) で、A=FE/7F の 1028 サイクルの約 4.4 倍。シフトと引き算による割り算なら商によらずほぼ一定になるので、置き換えた場合の効果は掛け算より大きい。

4.3 PRINT の書き込み位置の計算

PRINT は、書き込み位置を決めるために、Y をカーソルの値 +1 回だけ 1 ずつ進めている (C201 ~C209、9 バイト)。これは足し算 1 回で済む。

現行 (9 バイト)

```
CAL &11F9          ; Y=C7AF
P=&20 (LP)
A=(P) (LDM)
PUSH
{Y=Y+1,DP=Y} (IY)  ; カーソル+1回
LOOP &C206
{Y=Y-1,DP=Y} (DY)
```

案 (8 バイト)

```
P=&20 (LP)
A=(P) (LDM)          ; A=カーソル
A=A+&B0 (ADIA)
B=&C7 (LIB)
CAL &11F5          ; Y=BA-1
```

実 ROM で試すと表示は同じで、カーソルが 17(右端) のときに約 400 サイクル速くなった。ただし表示そのもの (ROM 11AF) が約 11200 サイクルかかるので、PRINT 全体では 3~4% 程度の差にしかない。1 バイト減るのが主な利点である。

4.4 ブロック転送命令の使いどころ

SC61860 には、1 命令で複数バイトを扱うブロック命令がある。このプログラムでの使用箇所を表 4.1 にまとめた。ブロック命令が使えらる場面では、すでに使われている。

表 4.1 SC61860 のブロック命令と、このプログラムでの使用箇所

命令	動作	使用箇所
[(DP)=A] (FILD)	外部メモリを A で埋める	COS、CLEAR、表示バッファ初期化、BEEP(遅延)
[(P)=(Q)] _{I/J} (MVW/MVB)	内部 RAM → 内部 RAM	FOR で X を保存、ROM の X 退避
[(P)=(DP)] _{I/J} (MVWD/MVBD)	外部メモリ → 内部 RAM	ROM 118F(X の設定)
[(P)↔(DP)] _{I/J} (EXWD/EXBD)	内部 RAM ↔ 外部メモリの交換	エラー時の X と C6B1 の交換
[(P)=A] (FILM)	内部 RAM を A で埋める	未使用

使えそうで使えないのが PRINT の文字列コピーである。SC61860 には「外部メモリから外部メモリへ」のブロック転送がない。しかも文字列の長さは閉じ"か行末を見つけるまで分からない。両方を調べて長さを数え (1 文字約 26 サイクル)、[(P)=(DP)]_I (MVWD) で内部 RAM へ (4 サイクル)、[(P)↔(DP)]_I (EXWD) で表示バッファへ (6 サイクル) と 2 段に運べば、固定費を除いて 1 文字約 36 サイクルになる。現行は 1 文字約 41 サイクルなので、差は 1 文字 5 サイクルほどである。そのうえ内部 RAM に文字列を置く場所が要り、コードも長くなる。PRINT で扱う文字列は短いことが多く、置き換える利点は少ない。

GOTO の行探しも 1 バイトずつ読んでいるが、SC61860 には「特定の値を探す」ブロック命令がないので、ここも置き換えられない (4.5 節)。

4.5 GOTO の行探しと行頭表

GOTO は毎回プログラムの先頭から 1 バイトずつ行末を探す。実測では、1 行 9 バイトの行を 1 行飛ばすごとに 155 サイクル (1 バイト約 17 サイクル) かかった。9 バイトの行を 50 行挟むと、GOTO の処理ルーチンに入ってから次の文を読み始めるまで約 8300 サイクル (43 ms) である。

起動時に 1 度だけプログラムを走査し、「行番号の下位 1 バイトと行頭の番地」の 3 バイトを 1 行

ずつ表に並べておけば、GOTO は表だけを見ればよい。表の 1 項目を調べるのは $\{A=(+X)\}$ (IXL)、 $(P)-A$ (CPMA)、JRP Z, n (JRZP)、 $\{X=X+1, DP=X\}$ (IX)2 回、JRM で約 33 サイクルなので、9 バイトの行を飛ばす部分なら約 5 倍、長い行ならそれ以上速くなる見込みである。

代償は、行数 $\times$ 3 バイトの RAM と、表を作るコード (30~40 バイト程度) である。PC-1245 のフリーエリアはこのインタープリタを入れると 510 バイトしか残らないので、PC-1245 では負担が大きい。PC-1251 専用にするなら効果が見込める。

4.6 IF のエラー番号

C2A0 の JP NZ, &C584 (JPNZ) は、IF の関係演算子が 4 種類のどれでもないときのエラー処理である。ところが飛び先の C584 はエラー番号 (14 番地) を設定しないので、IF A:B=01 のような誤りは「エラー 0」になる (エミュレータで確認)。

飛び先を文法エラー 1 を設定する C4F1 に変えて JP NZ, &C4F1 (JPNZ) とすれば、同じ 3 バイトのまま直る。実 ROM のエミュレータで、修正後はエラー 1 になり、正しい IF 文の動作は変わらないことを確かめた。

コラム 電卓の血筋

SC61860 には、BCD(10 進法の 1 桁を 4 ビットで表す 2 進化 10 進表記) を何バイトもまとめて足し引きする $[(P)=(P)+A]$ (ADN)、 $[(P)=(P)-A]$ (SBN)、 $[(P)=(P)+(Q)]$ (ADW)、 $[(P)=(P)-(Q)]$ (SBW) がある。utz82 によれば、内部 RAM の 10~2F 番地は BCD 演算用の作業域 (OP1~OP4) で、BASIC の浮動小数点計算はここで行われる。ポケットコンピュータが関数電卓から発展した機械であることが、命令セットにも表れている。

このインタープリタが計算結果を置く 10 番地は、この BCD 作業域の先頭にあたる。インタープリタは浮動小数点を使わないので、BASIC の計算用の領域を 1 バイトの「アキュムレータ」として借りている。

速さの感覚も今とは桁が違う。CPU のクロックは 192 kHz (MAME の設定) で、1 サイクルは約 $5.2 \mu\text{s}$ である。POPCOM の講座には外付けの水晶振動子が 576 kHz とあり、192 kHz はそのちょうど $1/3$ にあたる。記事の WAIT の時間と合うのは 192 kHz の方である。A=01 の 1 文は実測 617 サイクルで約 3.2 ms、1 秒に 310 文ほどしか実行できない。それでも BASIC より 5~7 倍速いというのが、この記事の売りだった。

コラム 同じ CPU を積んだ機種

POPCOM の講座は、SC61860 を「別名 ESR-H」と紹介し、PC-1240、1250、1260 の各シリーズで命令が共通だと書いている。MAME のポケットコンピュータのドライバ (pocketc.cpp) も、PC-1245、1250、1251、1255、1260、1261/1262、1350、1360、1401、1402、1403、1403H、1450 を同じ SC61860 で動かしている。クロックは PC-1403 だけが 256 kHz で、ほかは 192 kHz である。

米国では、PC-1251 が Tandy (Radio Shack) から TRS-80 Pocket Computer PC-3 として売られた。MAME でも PC-3 は PC-1250 の派生機として登録され、PC-1251 と同じ構成のドライバで扱われている。POPCOM の編集部がロサンゼルスの人を通じて手に入れたのも、米国で販売されていた英文のマニュアルだった。

機種ごとにメモリの配置も ROM の中身も違う。記事が対象とするのは PC-1245/51/55 で、PC-1251 と同じ構成の PC-3 でも動く見込みだが、確かめてはいない。

第 5 章

検証と残った疑問

5.1 記事と食い違う点・未確認の点

- 表 1 の「C2E5 LEFT\$」は C2E4 の 3 バイト命令 DP=&C6B1 (LIDP) の途中を指す。LEFT\$ の処理は見当たらず、誤植か旧版の名残と思われる。
- 表 1 の「C52F 式評価②」は分岐命令の位置で、区切りとしては C52B の方が自然である。
- FOR は値 n に対して $n + 1$ 回、WAIT も $n \geq 1$ では $n + 1$ 周になる (WAIT 00 は待たない)。記事の「指定した回数」とは 1 ずれる。
- BEEP は式 1 が大きいほど低い音になる。記事の説明とは逆である。
- 84 を <> とするのは推定である (ROM のキーワード表に文字列としては現れない)。
- ROM の確認は PC-1251 の ROM で行った。PC-1245 の ROM では確かめていない。

5.2 検証

表 1.3 の解釈で実装したエミュレータを 2 種類用意した。1 つは ROM ルーチンを推定どおりの簡易版に置き換えたもの、もう 1 つは PC-1251 の ROM を読み込み、キー入力以外の ROM ルーチンをそのまま実行するものである。結果を表 5.1 に示す。

表 5.1 エミュレータ (実 ROM 使用) による確認

試したこと	結果
A=03+04*02 ほか	0E(左から順)、FE+03 → 01、07/02 → 03
IF の 4 演算子と NOT	表 2.2 と図 2.9 のとおり
行 126 と 226 があるときの GOTO 26	行 126 へ飛ぶ
FOR 03	本体を 4 回実行
A=A+1、05/00、存在しない行への GOTO	エラー 1、2、4(14 番地)、キャリー付きで戻る
BASIC の CALL 文 (51C6) から実行	END は 668A へ戻り X も元どおり。エラーと BREAK は 6595 へ戻り、X と C6B1 はインタープリ タの実行位置を指す
WAIT の 1 単位	WAIT 11 と WAIT 10 の差は 748 サイクル (3.90 ms、 192 kHz)
BEEP の周波数	式 1=03, 09, 20 で約 2.1, 1.5, 0.7 kHz
表示	図 2.11 のとおり
記事のサンプルプログラム	最後まで実行され、正解キー 3 回で Z=3
4.2 節のシフト加算の掛け算	65536 通りすべて正しい
4.3 節の PRINT(8 バイト版)	カーソル 00, 05, 17 で表示が一致
4.6 節の IF 修正	エラー番号が 0 から 1 になり、正しい IF 文は変化 なし

これは「採用した命令解釈と ROM の動作のもとで、プログラムが記事どおりに実行される」こと
の確認であり、実機での確認ではない。

付録 A

命令一覧

このプログラムが使う 73 種類の命令を、POPCOM の講座の分類順に並べた。「回数」はプログラム中に現れる数である。動作の欄の n は 8 ビットの値、 nm は 16 ビットの番地を表す。† は講座に載っていない命令で、本書独自の表記である。

表 A.1: このプログラムが使う命令 (分類は POPCOM の講座による)

コード	BASIC 風命令記号	シャープ	動作	回数
代入				
00	I=n	LII	$I \leftarrow n$	5
02	A=n	LIA	$A \leftarrow n$	19
03	B=n	LIB	$B \leftarrow n$	7
12	P=n	LIP	$P \leftarrow n$	1
13	Q=n	LIQ	$Q \leftarrow n$	1
10	DP=nm	LIDP	$DP \leftarrow nm$	14
11	DPL=n	LIDL	DP の下位 $\leftarrow n$	1
80+ ℓ	P= ℓ	LP	$P \leftarrow \ell$ (0~3F、1 バイト命令)	44
32	R=A	STR	$R \leftarrow A$	3
59	A=(P)	LDM	$A \leftarrow (P)$	8
57	A=(DP)	LDD	$A \leftarrow (DP)$	3
52	(DP)=A	STD	$(DP) \leftarrow A$	4
55	(P)=(DP)	MVMD	$(P) \leftarrow (DP)$	1
53	(DP)=(P)	MVDM	$(DP) \leftarrow (P)$	4
56	A=(PC+1) [†]	READ	A ← 次の番地のバイト。そのバイトは命令として残る	2
交換				
DA	A↔B	EXAB	A と B を交換	12
DB	A↔(P)	EXAM	A と (P) を交換	22
58	AH↔AL	SWP	A の上位 4 ビットと下位 4 ビットを交換	2
加減算				
42	A=A+1	INCA	$A \leftarrow A+1$ (Q も変わる)	5
C2	B=B+1	INCB	$B \leftarrow B+1$ (Q も変わる)	1
50	P=P+1	INCP	$P \leftarrow P+1$	1

コード	BASIC 風命令記号	シャープ	動作	回数
41	I=I-1	DECI	$I \leftarrow I - 1$ (Q も変わる)	1
43	A=A-1	DECA	$A \leftarrow A - 1$ (Q も変わる)	8
C3	B=B-1	DECB	$B \leftarrow B - 1$ (Q も変わる)	1
74	A=A+n	ADIA	$A \leftarrow A + n$	1
70	(P)=(P)+n	ADIM	$(P) \leftarrow (P) + n$	4
44	(P)=(P)+A	ADM	$(P) \leftarrow (P) + A$	5
75	A=A-n	SBIA	$A \leftarrow A - n$	5
71	(P)=(P)-n	SBIM	$(P) \leftarrow (P) - n$	2
45	(P)=(P)-A	SBM	$(P) \leftarrow (P) - A$	6
論理演算				
60	(P)=(P).AND.n	ANIM	$(P) \leftarrow (P) \text{ AND } n$	3
46	(P)=(P).AND.A	ANMA	$(P) \leftarrow (P) \text{ AND } A$	2
47	(P)=(P).OR.A	ORMA	$(P) \leftarrow (P) \text{ OR } A$	1
比較				
67	A-n	CPIA	A - n でフラグだけ変える	56
63	(P)-n	CPIM	(P) - n でフラグだけ変える	5
C7	(P)-A	CPMA	(P) - A でフラグだけ変える	4
シフト				
D2	SHIFT R	SR	A をキャリーごと右へ 1 ビット回す	1
5A	SHIFT L	SL	A をキャリーごと左へ 1 ビット回す	1
ジャンプ				
2C	JRP n	JRP	前へ相対ジャンプ	10
38	JRP Z,n	JRZP	Z=1 なら前へ	16
28	JRP NZ,n	JRNZP	Z=0 なら前へ	12
3A	JRP C,n	JRCP	C=1 なら前へ	7
2A	JRP NC,n	JRNCP	C=0 なら前へ	5
2D	JRM n	JRM	後ろへ相対ジャンプ	4
39	JRM Z,n	JRZM	Z=1 なら後ろへ	9
29	JRM NZ,n	JRNZM	Z=0 なら後ろへ	1
3B	JRM C,n	JRCM	C=1 なら後ろへ	2
2B	JRM NC,n	JRNCM	C=0 なら後ろへ	4
79	JP nm	JP	nm へジャンプ	29
7E	JP Z,nm	JPZ	Z=1 なら nm へ	12
7C	JP NZ,nm	JPNZ	Z=0 なら nm へ	3
7F	JP C,nm	JPC	C=1 なら nm へ	4
7D	JP NC,nm	JPNC	C=0 なら nm へ	4
スタックと呼び出し				
34	PUSH	PUSH	A をスタックに積む	8
5B	POP	POP	スタックから A に取り出す	4
E0+ℓ	CAL ℓn	CAL	0000~1FFF を呼ぶ (2 バイト命令)	25
78	CALL nm	CALL	nm を呼ぶ	28
37	RETURN	RTN	呼び出し元へ戻る	15
複合				
04	{X=X+1,DP=X}	IX	$X \leftarrow X + 1$, $DP \leftarrow X$	9

コード	BASIC 風命令記号	シャープ	動作	回数
05	{X=X-1, DP=X}	DX	$X \leftarrow X - 1$, $DP \leftarrow X$	10
06	{Y=Y+1, DP=Y}	IY	$Y \leftarrow Y+1$, $DP \leftarrow Y$	3
07	{Y=Y-1, DP=Y}	DY	$Y \leftarrow Y - 1$, $DP \leftarrow Y$	1
24	{A=(+X)}	IXL	$X \leftarrow X+1$, $DP \leftarrow X$, $A \leftarrow (DP)$	17
25	{A=(-X)}	DXL	$X \leftarrow X - 1$, $DP \leftarrow X$, $A \leftarrow (DP)$	5
26	{(+Y)=A}	IYS	$Y \leftarrow Y+1$, $DP \leftarrow Y$, $(DP) \leftarrow A$	1
ブロック処理				
08	[(P)=(Q)] _I	MVW	(Q) から I+1 バイトを (P) へ	1
1F	[(DP)=A]	FIELD	DP 番地から I+1 バイトを A で埋める	6
19	[(P)↔(DP)] _I	EXWD	(P) と DP 番地の I+1 バイトを交換	1
その他				
D0	SET C	SC	$C \leftarrow 1$ (Z も 1)	2
4D	NOP	NOPW	何もしない (2 サイクル)	3
4E	WAIT n	WAIT	6+n サイクル待つ	3
2F	LOOP n	LOOP	$(R) \leftarrow (R) - 1$ 。ポローがなければ後ろへ、あればカウンタを POP	6
6B	TEST n	TEST	テストビットと n の AND で Z を変える	4

付録 B

注釈付き逆アセンブル・リスト

行頭の番地、機械語、命令の順に示す。命令は 1.2 節の BASIC 風命令記号で書き、括弧内にシャープのニーモニックを添えた。★は工夫している箇所、DB は実行されないバイトである。

```

; =====
; PC-インタープリタ(Fan_PC-1251 https://x.com/pio1986_10, PiO 1986年8月号p.169-171)
; 対象: SHARP PC-1245/1251/1255 (CPU: SC61860)
; 領域: C200-C5CF (976バイト)
; 起動: BASICからCALL &C300
;
; 逆アセンブルの手順
;   記事のリスト1を転記し、各行のチェックサム(8バイトの和の下位8ビット)が
;   すべて一致することを確認した。そのうえでC300と表1の各エントリから制御の
;   流れをたどって命令境界を決めた(再帰下降)。流れから到達しないバイトはDBで示す。
; 命令表: https://github.com/utz82/SC61860-Instruction-Setほか
;
; 表記
;   命令はPOPCOM 1984年9～11月号「PC-1250/1251マシン語講座」のBASIC風命令記号で書き、
;   括弧内にシャープのオリジナルニーモニックを添える(両者が同じ語なら省く)。
;   &付きの値は16進法。+は講座に載っていない命令で、本書独自の表記。
;   相対ジャンプとLOOPの行き先は、距離ではなく飛び先の番地(ラベル)で示す。
;   (P)は内部RAMのP番地、(DP)は外部メモリのDP番地の内容。{ }は1命令で行う複数の動作、
;   [ ]はブロック処理で、末尾のI/Jは長さ(I+1またはJ+1バイト)を決めるレジスタ。
;   即値アドレスはビッグエンディアン(78 C3 72 = CALL C372)。
;   {A=(+X)}(IXL)などはポインタを先に動かしてから読み書きする(プリインクリメント)。
;   そのためXは常に「最後に読んだバイト」を指す。
;   CAL xxxxは内部ROM(0000-1FFF)専用の2バイト呼び出し。ROMルーチンの役割は
;   PC-1251のROMを逆アセンブルして確かめた。
;
; 内部RAM(記事の表3と解析による)
;   04,05      : X(実行ポインタ)
;   10         : 計算結果(式評価のアクキュレータ)
;   12         : BEEP/DEGの作業
;   14         : エラーコード
;   15         : BASICの状態フラグ
;   19         : IFの関係演算子
;   1C,1D     : FORの戻り位置(Xの保存)
;   1E         : FORの残り回数
;   20         : カーソル位置/作業
;   3C,3D     : 乱数の種(BASICのワークと推定)
;
; RAM
;   C600-C69F : 配列(添字00-9F)
;   C686-C69F : 変数A-Z(各1バイト、配列と共用)
;   C6B1-C6B2 : BASICの実行位置の保存場所
;   C6DB-C6DD : CALL文が作るJP命令(トランポリン)
;   C6E1-C6E2 : プログラム先頭-1
;   C7B0-     : テキスト表示バッファ
;   F800-F83B, F840-F87B : LCD RAM
; =====
;
; -----
; PRINT "文字列"
;   ' 'の次から、閉じ' 'または行末(00)までをテキスト表示バッファへコピーし表示。
;   書き込み位置はreg20(カーソル、CALL文で設定)。変数の表示はできない(記事どおり)。
```

```

; IX/IYSはブラインクリメント型なので、ポインタは常に「最後に処理した番地」を指す。
; PRINT自身はカーソルを進めず、バッファも消さない(前の表示の残りが後ろに残る)。
; -----
CMD_PRINT:
C200 04      {X=X+1,DP=X} (IX)          ; X: PRINTトークン→開き"を飛ばす
C201 F1 F9   CAL &11F9                  ; ROM:Y←C7AF(テキストバッファ-1)
C203 A0      P=&20 (LP)                  ; A←カーソル位置
C204 59      A=(P) (LDM)
C205 34      PUSH                        ; ループ回数をスタックへ
C206 06      {Y=Y+1,DP=Y} (IY)          ; Yをカーソル+1個進める
C207 2F 02   LOOP &C206                 ; (R)-1,回数+1回まわる
C209 07      {Y=Y-1,DP=Y} (DY)          ; IYSはブラインクリメントなので1つ戻す
PRINT_LP:
C20A 24      {A=(+X)} (IXL)             ; 次の文字
C20B 67 12   A-&12 (CPIA)                ; 閉じ" ?
C20D 38 08   JRP Z,PRINT_SHOW (JRZP)
C20F 43      A=A-1 (DECA)                ; 00(行末)ならDECAでポロー発生
C210 3A A6   JRP C,PRINT_EOL (JRCP)
C212 42      A=A+1 (INCA)                ; 元の文字に戻す
C213 26      {(+Y)=A} (IYS)             ; バッファへ書き込み
C214 2D 0B   JRM PRINT_LP
PRINT_SHOW:
C216 F1 C7   CAL &11C7                  ; ROM:Xをスタックに積む
C218 F1 AF   CAL &11AF                  ; ROM:テキストバッファをLCDに表示
C21A F1 D2   CAL &11D2                  ; ROM:Xをスタックから戻す(Aは壊れる)
JP_NEXTSTMT:
C21C 79 C3 AA JP NEXT_STMT              ; ★SQRがJRMでここを借りる(3バイトJPの共用)
; -----
; CALL 式 ... reg20(カーソル位置)←式の値
; -----
CMD_CALL:
C21F 78 C3 72 CALL EVAL_A              ; A←式の値
C222 A0      P=&20 (LP)                  ; reg20(カーソル)←A
C223 DB      A↔(P) (EXAM)
C224 79 C3 A9 JP UNREAD_NEXT
; -----
; RND 式 (式評価①から呼ばれる関数) 0～式-1 の乱数を A に返す
; (1) n-1を覆う最小の2^k-1マスクを作る(C231～)
; (2) reg3C/3D(BASICの乱数用ワークと推定)と、TEST 02のハード信号が変わるまで
; 回したカウンタ値を混ぜて乱数を作り、マスクをかける
; (3) n-1を超えたらやり直す(棄却法)。元の乱数が一樣なら、剰余による偏りを避けられる。
; -----
FN_RND:
C227 78 C5 20 CALL OPERAND              ; A←上限n
C22A 80      P=&00 (LP)                  ; I←n
C22B DB      A↔(P) (EXAM)
C22C 41      I=I-1 (DECI)                ; I←n-1
C22D 59      A=(P) (LDM)                 ; A←n-1
C22E 03 FF   B=&FF (LIB)                 ; B←FF(マスク作業用)
C230 D0      SET C (SC)                  ; c←1
RND_MASK:
C231 DA      A↔B (EXAB)
C232 D2      SHIFT R (SR)                ; Bを右シフト(上位へcを入れる)
C233 DA      A↔B (EXAB)
C234 5A      SHIFT L (SL)                ; Aを左シフト
C235 2B 05   JRM NC,RND_MASK (JRNCM)     ; n-1の最上位の1が出るまで
RND_RETRY:
C237 BD      P=&3D (LP)                  ; reg3D(乱数ワーク)
C238 59      A=(P) (LDM)
C239 58      AH↔AL (SWP)
C23A 6B 02   TEST &02                   ; ハード入力ビット
RND_TICK:
C23C 42      A=A+1 (INCA)                ; 信号が変わるまでカウント
C23D 6B 02   TEST &02                   ; …これを種に混ぜる
C23F 39 04   JRM Z,RND_TICK (JRZM)
C241 44      (P)=(P)+A (ADM)              ; reg3D+=A
C242 BC      P=&3C (LP)
C243 70 71   (P)=(P)+&71 (ADIM)           ; reg3C+=71h
C245 59      A=(P) (LDM)
C246 BD      P=&3D (LP)
C247 44      (P)=(P)+A (ADM)              ; reg3D+=reg3C
C248 59      A=(P) (LDM)                 ; A←乱数
C249 DA      A↔B (EXAB)                 ; A←マスク, B←乱数
C24A 83      P=&03 (LP)
C24B 46      (P)=(P).AND.A (ANMA)         ; B&=マスク

```

```

C24C DA      A↔B (EXAB)                ; A←乱数&マスク
C24D 80      P=&00 (LP)                  ;
C24E C7      (P)-A (CPMA)                ; (n-1)-A
C24F 3B 19   JRM C,RND_RETRY (JRCM)      ; n-1を超えたら引き直し(棄却法)
C251 37      RETURN (RTN)
;
;-----
; SQR ... 画面(LCD RAM)を白黒反転
; LCDのRAMはF800-F83BとF840-F87Bの2ブロック(各60バイト)。
; NOT命令がないのでFF-A(SBM)でビット反転している。
; 2重ループはPUSHしたカウンタをLOOPで回すハードウェア・ループ。
;
;-----
CMD_SQR:
C252 03 F7   B=&F7 (LIB)                ; BA=F7FF
C254 02 FF   A=&FF (LIA)                  ;
C256 F1 EE   CAL &11EE                    ; ROM:Y←BA →Y
C258 02 01   A=&01 (LIA)                  ; 外側2回
C25A 34      PUSH
SQR_HALF:
C25B 02 3B   A=&3B (LIA)                ; 内側60回
C25D 34      PUSH
C25E 83      P=&03 (LP)                  ; P→B
SQR_BYTE:
C25F 06      {Y=Y+1,DP=Y} (IY)          ; DP←++Y
C260 57      A=(DP) (LDD)                ; A←LCD
C261 03 FF   B=&FF (LIB)                  ; B←FF
C263 45      (P)=(P)-A (SBM)             ; B←FF-A (=NOT A)
C264 53      (DP)=(P) (MVD)              ; LCD←B
C265 2F 07   LOOP SQR_BYTE
C267 86      P=&06 (LP)                  ; YL+=4: F83C-F83Fを飛ばしてF840へ
C268 70 04   (P)=(P)+&04 (ADIM)
C26A 2F 10   LOOP SQR_HALF
C26C 2D 51   JRM JP_NEXTSTMT            ; →JP NEXT_STMT
;
;-----
; IF 式1 関係演算子 式2 [THEN] 文 ...
; 式1と式2を比べ、AにFF(式1>式2)/00(等しい)/01(式1<式2)を作る。
; その後DECAのポロ- (キャリー)1本で真偽を判定するよう、演算子ごとにAを ±1ずらす。
; '=' : A-1でキャリー ⇔ A=00
; '<' : A-2でキャリー ⇔ A=01 (A=00→FFはキャリー無し…DECAを2回)
; '>' : A+1-1でキャリー ⇔ A=FF (INCAでFF→00にしてからDECA)
; '<>' : A-1でキャリー無し ⇔ A≠00
; 偽ならSKIP_TO_NOTで次のNOTが行末まで読み飛ばす(ELSE相当の仕掛け)。
;
;-----
CMD_IF:
C26E 78 C3 72 CALL EVAL_A                ; A←式1
C271 34      PUSH                        ; 式1を保存
C272 04      {X=X+1,DP=X} (IX)          ; 現在位置の文字を再読(IX→DXL)
C273 25      {A=(-X)} (DXL)
C274 99      P=&19 (LP)                  ; reg19←関係演算子
C275 DB      A↔(P) (EXAM)
C276 78 C3 72 CALL EVAL_A                ; A←式2
C279 DA      A↔B (EXAB)                ; B←式2
C27A 5B      POP                        ; A←式1
C27B 83      P=&03 (LP)
C27C C7      (P)-A (CPMA)                ; 式2-式1
C27D 3A 07   JRP C,IF_GT (JRCP)          ; 式1>式2
C27F 28 09   JRP NZ,IF_LT (JRNZP)        ; 式1<式2
C281 02 00   A=&00 (LIA)                  ; 等しい: 00
C283 2C 07   JRP IF_OP
IF_GT:
C285 02 FF   A=&FF (LIA)                  ; FF
C287 2C 03   JRP IF_OP
IF_LT:
C289 02 01   A=&01 (LIA)                  ; 01
IF_OP:
C28B 99      P=&19 (LP)
C28C 63 34   (P)-&34 (CPIM)              ; '='
C28E 38 1A   JRP Z,IF_TEST0 (JRZP)
C290 63 33   (P)-&33 (CPIM)              ; '<'
C292 28 04   JRP NZ,IF_NOT_LT (JRNZP)
C294 43      A=A-1 (DECA)                ; A-2相当に
C295 2C 13   JRP IF_TEST0
IF_NOT_LT:
C297 63 32   (P)-&32 (CPIM)              ; '>'
C299 28 04   JRP NZ,IF_NOT_GT (JRNZP)
C29B 42      A=A+1 (INCA)                ; A+1-1相当に

```

```

C29C 2C 0C      JRP IF_TEST0
IF_NOT_GT:
C29E 63 84      (P)-&84 (CPIM)          ; '<>'トークン(推定)
C2A0 7C C5 84   JP NZ,ERR_EXIT (JPNZ)    ; それ以外はエラー終了
C2A3 43         A=A-1 (DECA)
C2A4 7F C4 BD   JP C,SKIP_TO_NOT (JPC)    ; 等しければ偽
C2A7 2C 05      JRP IF_TRUE
IF_TEST0:
C2A9 43         A=A-1 (DECA)
C2AA 7D C4 BD   JP NC,SKIP_TO_NOT (JPNC)   ; ボロー無し=偽
IF_TRUE:
C2AD 04         {X=X+1,DP=X} (IX)         ; 真:次の文字が
C2AE 25         {A=(-X)} (DXL)
C2AF 67 92      A-&92 (CPIA)              ; THEN(92h,推定)なら読み捨て
C2B1 38 02      JRP Z,IF_EXEC (JRZP)
C2B3 05         {X=X-1,DP=X} (DX)        ; でなければ戻す
IF_EXEC:
C2B4 79 C3 0E   JP DISPATCH              ; IFの後ろの文を実行
;
; -----
; PRINT サブ: 行末で終わったとき。表示して、X を戻してから次の文へ
; -----
PRINT_EOL:
C2B7 F1 C7      CAL &11C7                ; ROM:Xをスタックに積む
C2B9 F1 AF      CAL &11AF                ; ROM:テキストバッファをLCDに表示
C2BB F1 D2      CAL &11D2                ; ROM:Xをスタックから戻す(Aは壊れる)
C2BD 79 C3 A9   JP UNREAD_NEXT
CMD_COS_TAIL:
C2C0 A0         P=&20 (LP)                ; reg20(カーソル)←0
C2C1 60 00      (P)=(P).AND.&00 (ANIM)
C2C3 4D         NOP (NOPW)               ; 詰め物(旧版の名残?)
C2C4 78 C2 D0   CALL TXTBUF_BLANK
C2C7 79 C3 AA   JP NEXT_STMT
;
; -----
; 初期設定: C6DA←0のあと、テキスト表示バッファC7B0-C7D0を21hで埋める。
; 21hはROMの表示ルーチンで空白として表示される(実ROMで確認)。
; C2D4から入ると00で埋める(終了時用)。
; -----
INIT:
C2CA 10 C6 DA   DP=&C6DA (LIDP)
C2CD 02 00      A=&00 (LIA)
C2CF 52         (DP)=A (STD)              ; (C6DA)←0
TXTBUF_BLANK:
C2D0 02 21      A=&21 (LIA)               ; 表示で空白になるコード
C2D2 2C 03      JRP TXTBUF_FILL
TXTBUF_ZERO:
C2D4 02 00      A=&00 (LIA)
TXTBUF_FILL:
C2D6 00 20      I=&20 (LII)               ; 33バイト
C2D8 10 C7 B0   DP=&C7B0 (LIDP)           ; テキスト表示バッファ
C2DB 1F         [(DP)=A] (FILD)
C2DC 37         RETURN (RTN)
;
; -----
; Ret 処理サブ (END): バッファを00で埋め、スタックを BASIC から CALL された時点(R=58)に戻して RTN
; -----
RET_BASIC:
C2DD 78 C2 D4   CALL TXTBUF_ZERO
C2E0 02 58      A=&58 (LIA)
C2E2 32         R=A (STR)                ; R←58: CALL &C300直後のスタック位置
C2E3 37         RETURN (RTN)             ; BASICへ戻る
;
; -----
; BASIC ヘエラー/BREAKで戻すための細工
; X(このインタープリタの実行ポインタ)とC6B1-C6B2を交換する。BASICのCALL文は
; 呼び出し前にXをC6B1に保存し(ROM 6787)、戻ったあとC6B1からXを戻す(ROM 678C)。
; 交換しておく、BASICは「インタープリタが実行していた位置」を自分の位置として受け取り、
; その行番号でエラーやBREAKを表示する。
; さらにreg15≠10なら、スタック奥(5A,5B)の戻り番地から下位F5・上位01を桁借りなしで引く。
; 文ディスパッチの戻り番地668Aが6595(もう1つのディスパッチ箇所の戻り番地)に変わり、
; BASICはCALLの次の文へ進まずに止まる(実ROMで確認)。
; 表1の"C2E5 LEFT$"はこの命令の途中を指しており、誤植か旧版の名残と思われる。
; -----
FIX_BASIC_CTX:
C2E4 10 C6 B1   DP=&C6B1 (LIDP)           ; BASICがCALL前にXを保存した場所
C2E7 84         P=&04 (LP)
C2E8 00 01      I=&01 (LII)
C2EA 19         [(P)↔(DP)]I (EXWD)       ; X ↔ (C6B1,C6B2)

```

```

C2EB 95      P=&15 (LP)
C2EC 63 10   (P)-&10 (CPIM)          ; reg15=10 ?
C2EE 38 08   JRP Z, FIX_RTN (JRZP)
C2F0 12 5A   P=&5A (LIP)
C2F2 71 F5   (P)=(P)-&F5 (SBIM)      ; 文ディスパッチへの戻り番地(668A)
C2F4 50      P=P+1 (INCP)            ; 下位-F5
C2F5 71 01   (P)=(P)-&01 (SBIM)      ; 上位-01(ポロー無し)
FIX_RTN:
C2F7 37      RETURN (RTN)
;
; ★未使用バイト:直前のC2F2-C2F7と同じ命令列+2バイト。改版時の残骸と思われる
;
C2F8 71 F5 50 71 01 37 08 12 DB
;
; メイン・ルーチン(BASICからCALL &C300で入る)
; BASICのプログラム領域を「先頭から」自前の文法で解釈実行する。
; → 行の入力・編集・中間コード化(トークン化)はBASICのエディタにまかせ、
; インタープリタは中間コードを読むだけ、という設計。
; このインタープリタが仮定している行の形式は次のとおり。
; [E0+百の位] [下2桁(BCD)] [中間コード...] [00]、プログラム終端はFF
; (行長バイトを読まずに00まで走査しているので、行長バイトは無い前提)
;
MAIN:
C300 F1 8F   CAL &118F              ; ROM:X←(C6E1,C6E2)プログラム先頭-1
C302 F1 E0   CAL &11E0              ; ROM:ポートC←01
C304 78 C5 9A CALL INIT_FOR          ; reg1E←0,初期化
NEXT_LINE:
C307 24      {A=(+X)} (IXL)          ; 行番号上位
C308 67 FF   A-&FF (CPIA)            ; FF=プログラム終端
CHK_END:
C30A 7E C5 80 JP Z,END_PROG (JPZ)
SKIP_LNLO:
C30D 04      {X=X+1,DP=X} (IX)       ; 行番号下位を読み飛ばす
;
; コマンド判断:文の先頭バイトを比較の連鎖で振り分ける
; SC61860にはテーブルジャンプ(PTC/DTC)もあるが、ここではCPIAと条件ジャンプの連鎖を使っている。
; 近い飛び先は2バイトの相対ジャンプJRZP/JRZM、遠い先は3バイトのJPZと使い分けている。
; どれにも当てはまらなければ最後に変数代入とみなす。
;
DISPATCH:
C30E 24      {A=(+X)} (IXL)          ; 文の先頭
C30F 6B 08   TEST &08              ; BRKキー?
C311 7C C5 8F JP NZ,BRK_EXIT (JPNZ)
C314 67 FF   A-&FF (CPIA)            ; FF
C316 39 0D   JRM Z,CHK_END (JRZM)
C318 67 D2   A-&D2 (CPIA)            ; D2h: LET → 無視して次へ
C31A 39 0D   JRM Z,DISPATCH (JRZM)
C31C 67 87   A-&87 (CPIA)            ; 87h: SQR
C31E 39 CD   JRM Z,CMD_SQR (JRZM)
C320 67 D0   A-&D0 (CPIA)            ; D0h: IF
C322 39 B5   JRM Z,CMD_IF (JRZM)
C324 67 C1   A-&C1 (CPIA)            ; C1h: PRINT
C326 7E C2 00 JP Z,CMD_PRINT (JPZ)
C329 67 C9   A-&C9 (CPIA)            ; C9h: CALL
C32B 7E C2 1F JP Z,CMD_CALL (JPZ)
C32E 67 86   A-&86 (CPIA)            ; 86h: NOT
C330 7E C4 BD JP Z,SKIP_TO_NOT (JPZ)
C333 67 D4   A-&D4 (CPIA)            ; D4h: END ★Z=1のままC30AのJPZに飛び込む
C335 39 2C   JRM Z,CHK_END (JRZM)
C337 67 DB   A-&DB (CPIA)            ; DBh: BEEP
C339 38 F5   JRP Z,CMD_BEEP (JRZP)
C33B 67 95   A-&95 (CPIA)            ; 95h: WAIT
C33D 38 C2   JRP Z,CMD_WAIT (JRZP)
C33F 67 C5   A-&C5 (CPIA)            ; C5h: CLEAR
C341 38 E3   JRP Z,CMD_CLEAR (JRZP)
C343 67 A1   A-&A1 (CPIA)            ; A1h: COS
C345 38 D2   JRP Z,CMD_COS (JRZP)
C347 67 D7   A-&D7 (CPIA)            ; D7h: GOTO
C349 7E C4 7B JP Z,CMD_GOTO (JPZ)
C34C 67 12   A-&12 (CPIA)            ; 12h: " (REM相当)
C34E 7E C4 BD JP Z,SKIP_TO_NOT (JPZ)
C351 67 D5   A-&D5 (CPIA)            ; D5h: NEXT
C353 7E C4 C9 JP Z,CMD_NEXT (JPZ)
C356 67 D1   A-&D1 (CPIA)            ; D1h: FOR
C358 7E C4 D8 JP Z,CMD_FOR (JPZ)

```

```

C35B 67 D6      A-&D6 (CPIA)                ; D6h: STOP
C35D 38 F0      JRP Z,CMD_STOP (JRZP)
C35F 67 AC      A-&AC (CPIA)                ; ACh: DEG
C361 38 F5      JRP Z,CMD_DEG (JRZP)
C363 67 89      A-&89 (CPIA)                ; 89h: COM$
C365 28 0A      JRP NZ,DISP_VAR (JRNZP)
C367 F1 C7      CAL &11C7                  ; ROM:Xをスタックに積む
C369 78 C5 D0    CALL USR_EXT                ; ユーザー拡張C5D0を呼ぶ
C36C F1 D2      CAL &11D2                  ; ROM:Xをスタックから戻す(Aは壊れる)
C36E 2C 3B      JRP NEXT_STMT
DISP_VAR:
C370 2C 07      JRP CMD_LET
; -----
; EVAL_A: 式を評価して結果を A に返す (EVAL は reg10 に残すので EXAM で取り出す)
; -----
EVAL_A:
C372 78 C5 00    CALL EVAL
C375 90          P=&10 (LP)                  ; A←reg10: A←結果
C376 DB          A↔(P) (EXAM)
C377 37          RETURN (RTN)
; -----
; 変数代入 変数[] [=]式
; 変数A〜Z(51h〜6Ah)はC686〜C69Fの各1バイト。配列はC600+添字(添字は変数でのみ指定: "A)=24")。
; ★自己書き換え:格納先番地の下位バイトを、後続のLIDP命令のオペランドへ直接書き込む。
; -----
CMD_LET:
C378 67 51      A-&51 (CPIA)                ; 'A'(51h)未滿はエラー
C37A 7F C4 F1    JP C,ERR1 (JPC)
C37D 67 6B      A-&6B (CPIA)                ; 'Z'(6Ah)を超えたらエラー
C37F 7D C4 F1    JP NC,ERR1 (JPNC)
LET_VAR:
C382 74 35      A=A+&35 (ADIA)              ; 51h+35h=86h: C686の下位
C384 DA          A↔B (EXAB)                ; B←変数番地下位
C385 24          {A=(+X)} (IXL)            ; 次の文字
C386 67 31      A-&31 (CPIA)                ; ')'なら配列(A=...)
C388 28 0C      JRP NZ,LET_EQ (JRNZP)
C38A 10 C3 91    DP=&C391 (LIDP)             ; ★C391 =下のLIDPの下位オペランド
C38D DA          A↔B (EXAB)
C38E 52          (DP)=A (STD)               ; 自己書き換え
C38F 10 C6 86    DP=&C686 (LIDP)            ; LIDP C6xx (xxは書き換え済み)
C392 57          A=(DP) (LDD)              ; A←変数の値
C393 DA          A↔B (EXAB)                ; B←値(=配列の添字 → C600+値)
C394 04          {X=X+1,DP=X} (IX)         ; 下のDXと相殺
LET_EQ:
C395 05          {X=X-1,DP=X} (DX)         ; 読んだ文字を戻す
C396 24          {A=(+X)} (IXL)
C397 67 34      A-&34 (CPIA)                ; '='なら消費
C399 38 02      JRP Z,LET_STORE (JRZP)
C39B 05          {X=X-1,DP=X} (DX)         ; なければ戻す(=は省略可)
LET_STORE:
C39C 10 C3 A6    DP=&C3A6 (LIDP)             ; ★C3A6 =下のLIDPの下位オペランド
C39F DA          A↔B (EXAB)
C3A0 52          (DP)=A (STD)               ; 自己書き換え
C3A1 78 C5 00    CALL EVAL                  ; 式を評価→reg10
C3A4 10 C6 88    DP=&C688 (LIDP)            ; LIDP C6xx (書き換え済み)
C3A7 90          P=&10 (LP)
C3A8 53          (DP)=(P) (MVDM)           ; 変数←reg10
; -----
; 文の区切り処理(ほぼ全コマンドがここへ戻る)
; C3A9から入るとXを1つ戻してから(=最後に読んだ文字をもう一度読む)判定する。
; ':(1Dh) → 次の文をDISPATCH
; 00 → 行末。NEXT_LINEへ(次の行番号を読む)
; その他 → Xを戻して、その文字から次の文としてDISPATCH (':'を省略できる理由)
; -----
UNREAD_NEXT:
C3A9 05          {X=X-1,DP=X} (DX)         ; 最後に読んだ文字をもう一度
NEXT_STMT:
C3AA 24          {A=(+X)} (IXL)
C3AB 67 1D      A-&1D (CPIA)                ; ':'
C3AD 7E C3 0E    JP Z,DISPATCH (JPZ)
C3B0 43          A=A-1 (DECA)              ; 00?
C3B1 3B AB      JRM C,NEXT_LINE (JRCM)     ; 行末→次の行
C3B3 05          {X=X-1,DP=X} (DX)         ; 戻して
C3B4 79 C3 0E    JP DISPATCH              ; そのまま次の文として実行
; -----

```

```

; 未使用00
;
-----
C3B7 00 DB
-----
;
; 四則演算処理(EVAL_LPから、演算子らしき文字Aを持って来る)
; 括弧も優先順位も無く、左から順にreg10に対して演算する。オーバーフローは無視(8ビット)。
; 演算子以外(35h~38h以外)ならRTNで式の終わり。
;
-----
ARITH:
C3B8 67 39 A-&39 (CPIA)
C3BA 2A 05 JRP NC,ARITH_END (JRNCP) ; 39h以上は演算子でない
C3BC 67 35 A-&35 (CPIA)
C3BE 2A 02 JRP NC,ARITH_ADD (JRNCP) ; 35h-38hが+ - * /
ARITH_END:
C3C0 37 RETURN (RTN) ; 式の終わり
ARITH_ADD:
C3C1 67 35 A-&35 (CPIA)
C3C3 28 09 JRP NZ,ARITH_SUB (JRNZP)
C3C5 78 C5 20 CALL OPERAND ; 右オペランド
C3C8 90 P=&10 (LP)
C3C9 44 (P)=(P)+A (ADM) ; reg10+=A
C3CA 79 C5 05 JP EVAL_LP ; 続きの演算子へ
ARITH_SUB:
C3CD 67 36 A-&36 (CPIA)
C3CF 28 09 JRP NZ,ARITH_MUL (JRNZP)
C3D1 78 C5 20 CALL OPERAND
C3D4 90 P=&10 (LP)
C3D5 45 (P)=(P)-A (SBM) ; reg10-=A
C3D6 79 C5 05 JP EVAL_LP
;
; 乗算:小さい方を回数にして加算を繰り返す。
; LOOPは(カウンタ+1)回まわるので、最後に1回分SBMで引いて帳尻を合わせる。
;
-----
ARITH_MUL:
C3D9 67 37 A-&37 (CPIA)
C3DB 28 15 JRP NZ,ARITH_DIV (JRNZP)
C3DD 78 C5 20 CALL OPERAND
C3E0 90 P=&10 (LP)
C3E1 C7 (P)-A (CPMA) ; reg10とAを比べ
C3E2 2A 02 JRP NC,MUL_GO (JRNCP)
C3E4 DB A↔(P) (EXAM) ; Aが小さい方になるよう交換
MUL_GO:
C3E5 34 PUSH ; 回数=小さい方
C3E6 59 A=(P) (LDM) ; A←大きい方
C3E7 60 00 (P)=(P).AND.&00 (ANIM) ; reg10←0
C3E9 4D NOP (NOPW) ; 詰め物
MUL_LP:
C3EA 44 (P)=(P)+A (ADM) ; reg10+=Aを 回数+1回
C3EB 2F 02 LOOP MUL_LP
C3ED 45 (P)=(P)-A (SBM) ; 1回多い分を引く
C3EE 79 C5 05 JP EVAL_LP
;
; 除算:引けなくなるまで引いた回数を数える(引き放し法)。除数0はERR2。
;
-----
ARITH_DIV:
C3F1 78 C4 A0 CALL GET_DIVISOR
C3F4 03 00 B=&00 (LIB) ; B=商
C3F6 90 P=&10 (LP)
DIV_LP:
C3F7 C2 B=B+1 (INCB)
C3F8 45 (P)=(P)-A (SBM) ; reg10=除数
C3F9 2B 03 JRM NC,DIV_LP (JRNCM) ; 引けるあいだ繰り返す
C3FB C3 B=B-1 (DECB) ; 引きすぎた1回分
C3FC DA A↔B (EXAB) ; A←商
C3FD 79 C5 03 JP EVAL_STORE ; reg10←商 として続行
;
; WAIT 式 ... 式*約1/256秒 待つ
; 内側のWAIT C8/FF/FFとTEST/JPNZ/LOOPを合計すると約748サイクル(utz82のサイクル表)。
; CPUクロック192kHz(MAMEの設定)なら約3.9ms≒1/256秒で、記事の「n/256秒」と合う。
; ループ中もBRKキーを見ている。
;
-----
CMD_WAIT:
C400 78 C3 72 CALL EVAL_A ; A←待ち時間
C403 34 PUSH ; 回数をスタックへ
C404 67 00 A-&00 (CPIA) ; 0なら待たない

```

```

C406 38 07      JRP Z, WAIT_CHK (JRZP)
WAIT_LP:
C408 4E C8      WAIT &C8                      ; 約1/256秒
C40A 4E FF      WAIT &FF
C40C 4E FF      WAIT &FF
WAIT_CHK:
C40E 6B 08      TEST &08                      ; BRK?
C410 7C C5 8F   JP NZ, BRK_EXIT (JPNZ)
C413 2F 0C      LOOP WAIT_LP                  ; (R)-1
C415 79 C3 A9   JP UNREAD_NEXT
;
; COS ... 画面クリア (CLS)
; ★ LDPC (=READ, 56h。次の番地の1バイトをAに読み、そのバイトは消費しない)で、直後の
; LII命令のオペコード00をAに読み込んでいる。LIA 00(2バイト)より1バイト短い。
; 資料によっては56hを2バイト命令としているが、それではこの部分が働かない。
;
CMD_COS:
C418 10 F8 00   DP=&F800 (LIDP)                ; LCD前半
C41B 56         A=(PC+1) ↑ (READ)              ; ★A←(C41C)=00 (LIIのオペコード)
C41C 00 3B      I=&3B (LII)                    ; 60バイト
C41E 1F         [(DP)=A] (FILD)                ; F800-F83B←0
C41F 11 40      DPL=&40 (LIDL)                 ; DP←F840
C421 1F         [(DP)=A] (FILD)                ; F840-F87B←0
C422 79 C2 C0   JP CMD_COS_TAIL               ; カーソル0とテキストバッファのクリアへ
;
; CLEAR ... C600-C69F(配列と固定変数 A~Z)を 0 に。これも LDPC で A=00 を作る
;
CMD_CLEAR:
C425 10 C6 00   DP=&C600 (LIDP)                ; 配列/変数領域
C428 56         A=(PC+1) ↑ (READ)              ; ★A←(C429)=00
C429 00 9F      I=&9F (LII)                    ; 160バイト
C42B 1F         [(DP)=A] (FILD)                ; C600-C69F←0
C42C 79 C3 AA   JP NEXT_STMT
;
; BEEP 式1(高さ), 式2(長さ)
; ★遅延ループの代わりに「ROM番地0000へのFILD」を使う。ROMなので書き込みは無害で、
; 所要時間だけがI(=式1)に比例する(FILDは4+3×Iサイクル)。
; ROM 1FB1はAをポートC(OUTC)に出す。51hと41hはビット4だけが違い、圧電ブザーを直接振る。
; 式1が大きいほど半周期が長く、音は低くなる(記事の説明とは逆)。
;
CMD_BEEP:
C42F 78 C3 72   CALL EVAL_A                   ; A←式1(高さ)
C432 92         P=&12 (LP)                     ; reg12←式1
C433 DB        A↔(P) (EXAM)
C434 78 C3 72   CALL EVAL_A                   ; A←式2(長さ)
C437 34         PUSH                          ; 回数
C438 92         P=&12 (LP)                     ; A←式1
C439 DB        A↔(P) (EXAM)
C43A 80         P=&00 (LP)                     ; I←式1
C43B DB        A↔(P) (EXAM)
BEEP_LP:
C43C 10 00 00   DP=&0000 (LIDP)                ; ★ROM 0000番地へFILD = Iに比例する空回し
C43F 02 51      A=&51 (LIA)                    ; 出力値1
C441 FF B1      CAL &1FB1                      ; ROM:A→(5F)→ポートC(OUTC)
C443 1F         [(DP)=A] (FILD)                ; 遅延
C444 02 41      A=&41 (LIA)                    ; 出力値2
C446 FF B1      CAL &1FB1                      ; ROM:A→(5F)→ポートC(OUTC)
C448 1F         [(DP)=A] (FILD)                ; 遅延
C449 2F 0E      LOOP BEEP_LP                  ; 式2+1周期
C44B 79 C3 A9   JP UNREAD_NEXT
;
; STOP ... キー入力待ってから続行
;
CMD_STOP:
C44E F1 C7      CAL &11C7                      ; ROM:Xをスタックに積む
C450 FD 0D      CAL &1D0D                      ; ROM:キー入力待ち
C452 F1 D2      CAL &11D2                      ; ROM:Xをスタックから戻す(Aは壊れる)
C454 79 C3 AA   JP NEXT_STMT
;
; DEG 位置, パターン, パターン, ... LCD の縦1列(7ドット)に直接パターンを書く
; 位置00~3BはF800~ の順、3C~77はF840~F87Bに「逆順」で並ぶのでB7-位置 で変換している。
;
CMD_DEG:
C457 78 C3 72   CALL EVAL_A                   ; A←位置
C45A 92         P=&12 (LP)                     ; reg12←位置

```

```

C45B DB      A↔(P) (EXAM)
DEG_LP:
C45C 59      A=(P) (LDM)           ; A←位置
C45D 67 3C   A=&3C (CPIA)          ; 前半(00-3B)?
C45F 3A 06   JRP C,DEG_ADR (JRCP)
C461 83      P=&03 (LP)            ; P→B
C462 03 B7   B=&B7 (LIB)           ;
C464 45      (P)=(P)-A (SBM)       ; B←B7-位置(後半は逆順)
C465 DA      A↔B (EXAB)           ; A←番地下位
DEG_ADR:
C466 03 F8   B=&F8 (LIB)           ; 上位F8
C468 F1 F5   CAL &11F5             ; ROM:Y←BA-1 Y←F8xx系
C46A 78 C5 00 CALL EVAL           ; パターンを評価→reg10 (P=10のまま戻る)
C46D 06      {Y=Y+1,DP=Y} (IY)
C46E 53      (DP)=(P) (MVDM)       ; LCD←パターン
C46F 92      P=&12 (LP)
C470 70 01   (P)=(P)+&01 (ADIM)    ; 位置+1
C472 04      {X=X+1,DP=X} (IX)    ; 現在の文字を再読
C473 25      {A=(-X)} (DXL)
C474 67 1B   A=&1B (CPIA)          ; ', 'なら次のパターン
C476 39 1B   JRM Z,DEG_LP (JRZM)
C478 79 C3 A9 JP UNREAD_NEXT
; -----
; GOTO 式
; 式の値(1バイト)を「行番号の下位バイト(BCD)」と比べ、先頭の行から探す。
; 行番号はBCDで格納されているので、GOTO 26で26/126/426…の最初の一つに飛ぶ。
; 行を飛ばすときは00(行末)まで1バイトずつ読み飛ばす(行長バイトを使っていない)。
; →記事の「GOTOは遅い」「よく使うサブルーチンは先頭に」の理由。
; -----
CMD_GOTO:
C47B 78 C5 00 CALL EVAL           ; reg10←行番号下位
C47E F1 C7   CAL &11C7             ; ROM:Xをスタックに積む
C480 F1 8F   CAL &118F             ; ROM:X←(C6E1,C6E2)プログラム先頭-1
C482 90      P=&10 (LP)            ; P→reg10
C483 2C 05   JRP GOTO_LINE
GOTO_SKIP:
C485 24      {A=(+X)} (IXL)        ; 行末(00)まで1バイトずつ読み飛ばす
C486 43      A=A-1 (DECA)
C487 2B 03   JRM NC,GOTO_SKIP (JRNCM)
GOTO_LINE:
C489 24      {A=(+X)} (IXL)        ; 行番号上位
C48A 67 FF   A=&FF (CPIA)          ; FF=終端:行が無い
C48C 38 0A   JRP Z,GOTO_ERR (JRZP)
C48E 24      {A=(+X)} (IXL)        ; 行番号下位(BCD)
C48F C7      (P)-A (CPMA)          ; 目的の値と比較
C490 29 0C   JRM NZ,GOTO_SKIP (JRNZM) ; 違えば次の行へ
C492 5B      POP                   ; 退避したXを捨てる
C493 5B      POP
C494 79 C3 0E JP DISPATCH         ; 見つかった行を実行
GOTO_ERR:
C497 F1 D2   CAL &11D2             ; ROM:Xをスタックから戻す(Aは壊れる) XをGOTO文の位置に戻す
C499 02 04   A=&04 (LIA)           ; エラー4
C49B 94      P=&14 (LP)
C49C DB      A↔(P) (EXAM)
C49D 79 C5 84 JP ERR_EXIT
; -----
; 除数を取得し、0ならERR2 (除算エラー処理)
; -----
GET_DIVISOR:
C4A0 78 C5 20 CALL OPERAND        ;
C4A3 67 00   A=&00 (CPIA)          ; 除数0?
C4A5 38 52   JRP Z,ERR2 (JRZP)
C4A7 37      RETURN (RTN)
; -----
; PEEK 式 … LCD RAM の内容を読む。DEG と同じ位置変換。
; ★自己書き換え:計算した下位バイトを後続のLIDP F8xxのオペランドに書き込んでからLDD。
; -----
FN_PEEK:
C4A8 78 C5 20 CALL OPERAND        ; A←位置
C4AB 67 3C   A=&3C (CPIA)          ; 前半?
C4AD 3A 06   JRP C,PEEK_ADR (JRCP)
C4AF 83      P=&03 (LP)
C4B0 03 B7   B=&B7 (LIB)
C4B2 45      (P)=(P)-A (SBM)       ; B←B7-位置
C4B3 DA      A↔B (EXAB)

```

```

PEEK_ADR:
C4B4 10 C4 BA DP=&C4BA (LIDP) ; ★C4BA =下のLIDPの下位
C4B7 52 (DP)=A (STD) ; 自己書き換え
C4B8 10 F8 3B DP=&F83B (LIDP) ; LIDP F8xx (書き換え済み)
C4BB 57 A=(DP) (LDD) ; A←LCD
C4BC 37 RETURN (RTN)
;
; '"" 文 / NOT / IF 偽: 次の NOT(86h) が行末(00) まで読み飛ばす
; NOTが見つければその後ろから実行を再開する。
; IF ... : NOT ... : NOT ... で「成立時は②④、不成立時は③⑤」になるのは、
; 実行中にNOTに出会うと次のNOTまで飛ばすという、このルーチンの共用による。
; '""で始まる行(BASICの"="などのラベル行)もこれで読み飛ばされる。
;
SKIP_TO_NOT:
C4BD 24 {A=(+X)} (IXL)
C4BE 67 86 A-&86 (CPIA) ; NOT?
C4C0 38 05 JRP Z,SKIP_DONE (JRZP)
C4C2 43 A=A-1 (DECA) ; 00?
C4C3 2B 07 JRM NC,SKIP_TO_NOT (JRNCM)
C4C5 05 {X=X-1,DP=X} (DX) ; 行末:戻して
SKIP_DONE:
C4C6 79 C3 AA JP NEXT_STMT
;
; NEXT: reg1E(回数)を1減らし、0 未満にならないければ reg1C/1D に保存した位置へ戻る
; カウンタも戻り位置も1組しかないのでFOR〜NEXTは1段のみ(記事P.S.のとおり)。
; コード上はFOR nで本体がn+1回実行される。
;
CMD_NEXT:
C4C9 9E P=&1E (LP) ; reg1E: FORの残り回数
C4CA 59 A=(P) (LDM)
C4CB 43 A=A-1 (DECA)
C4CC 2A 06 JRP NC,NEXT_LOOP (JRNCP) ; まだ残っていれば戻る
C4CE 42 A=A+1 (INCA) ; 00に戻して
C4CF DB A↔(P) (EXAM) ; reg1E←0
C4D0 79 C3 AA JP NEXT_STMT ; ループを抜けて続行
NEXT_LOOP:
C4D3 DB A↔(P) (EXAM) ; reg1E←回数-1
C4D4 F8 99 CAL &1899 ; ROM:X←(1C,1D)
C4D6 2C 0C JRP FOR_END
;
; FOR 式: reg1E←回数、reg1C/1D←X(現在の実行位置)を MVW で保存
;
CMD_FOR:
C4D8 78 C3 72 CALL EVAL_A ; A←回数
C4DB 9E P=&1E (LP) ; reg1E←回数
C4DC DB A↔(P) (EXAM)
C4DD 9C P=&1C (LP) ; P=1C
C4DE 13 04 Q=&04 (LIQ) ; Q=04 (X)
C4E0 00 01 I=&01 (LII) ; 2バイト
C4E2 08 [(P)=(Q)]I (MVW) ; (1C,1D)←X
FOR_END:
C4E3 79 C3 A9 JP UNREAD_NEXT
;
; ★未使用: CALL INIT / LIA 00 / LP 1E / EXAM / RTN。INIT_FOR(C59A)の旧版と思われる
;
C4E6 78 C2 CA 02 00 9E DB 37 DB
C4EE 00 00 00 DB
;
; エラー: reg14にエラーコードを入れてERR_EXITへ
;
ERR1:
C4F1 02 01 A=&01 (LIA) ; エラー1(文法)
SET_ERR:
C4F3 94 P=&14 (LP) ; reg14←エラーコード
C4F4 DB A↔(P) (EXAM)
C4F5 79 C5 84 JP ERR_EXIT
ERR2:
C4F8 02 02 A=&02 (LIA) ; エラー2(演算)
C4FA 79 C4 F3 JP SET_ERR
;
; E(指数記号 4Bh) は定数 01 として扱う
;
CONST_E:
C4FD 02 01 A=&01 (LIA)
C4FF 37 RETURN (RTN)

```

```

; -----
; 式評価サブルーチン・コール
; 値[演算子 値]...結果はreg10。AND(81h)/OR(85h)はここで、+ - * /はARITHで処理。
; -----
EVAL:
C500 78 C5 20 CALL OPERAND ; 1つ目のオペランド
EVAL_STORE:
C503 90 P=&10 (LP) ; reg10←A
C504 DB A↔(P) (EXAM)
EVAL_LP:
C505 24 {A=(+X)} (IXL) ; 次の文字
C506 67 81 A-&81 (CPIA) ; AND(81h)?
C508 28 08 JRP NZ,EVAL_OR (JRNZP)
C50A 78 C5 20 CALL OPERAND
C50D 90 P=&10 (LP)
C50E 46 (P)=(P).AND.A (ANMA) ; reg10&=A
C50F 2D 0B JRM EVAL_LP
EVAL_OR:
C511 67 85 A-&85 (CPIA) ; OR(85h)?
C513 28 08 JRP NZ,EVAL_ARITH (JRNZP)
C515 78 C5 20 CALL OPERAND
C518 90 P=&10 (LP)
C519 47 (P)=(P).OR.A (ORMA) ; reg10|=A
C51A 2D 16 JRM EVAL_LP
EVAL_ARITH:
C51C 79 C3 B8 JP ARITH ; + - * /か式の終わり
; -----
; 未使用00
; -----
C51F 00 DB
; -----
; 式評価① (オペランド1つをAに返す)
; PEEK(AFh) / RND(AEh) / INKEY$(8Ah) / "c"(文字コード) / E(=01) / &hh・hh(16進2桁) / 変数/変数)
; -----
OPERAND:
C520 24 {A=(+X)} (IXL)
C521 67 AF A-&AF (CPIA) ; PEEK(AFh)?
C523 7E C4 A8 JP Z,FN_PEEK (JPZ)
C526 67 AE A-&AE (CPIA) ; RND(AEh)?
C528 7E C2 27 JP Z,FN_RND (JPZ)
OPERAND2:
C52B 05 {X=X-1,DP=X} (DX) ; 戻して
C52C 24 {A=(+X)} (IXL) ; 再読
C52D 67 8A A-&8A (CPIA) ; INKEY$(8Ah)?
C52F 28 0C JRP NZ,OPD_CHR (JRNZP)
; -----
; INKEY$ ... ROM のキースキャンを呼ぶ。X を退避・復帰する ROM が A を壊すので reg10 経由で守る
; -----
FN_INKEY:
C531 F1 C7 CAL &11C7 ; ROM:Xをスタックに積む
C533 FF 44 CAL &1F44 ; ROM:キースキャン→A(BRKなら07) A←キーコード
C535 90 P=&10 (LP) ; Aをreg10に避難
C536 DB A↔(P) (EXAM)
C537 F1 D2 CAL &11D2 ; ROM:Xをスタックから戻す(Aは壊れる)
C539 90 P=&10 (LP) ; A←キーコード
C53A DB A↔(P) (EXAM)
C53B 37 RETURN (RTN)
OPD_CHR:
C53C 67 12 A-&12 (CPIA) ; ' '?
C53E 28 04 JRP NZ,OPD_E (JRNZP)
C540 24 {A=(+X)} (IXL) ; A←文字コード
C541 04 {X=X+1,DP=X} (IX) ; 閉じ"を飛ばす
C542 37 RETURN (RTN)
OPD_E:
C543 67 4B A-&4B (CPIA) ; E(4Bh)?
C545 39 49 JRM Z,CONST_E (JRZM)
C547 67 1F A-&1F (CPIA) ; '&'(1Fh)?
C549 38 02 JRP Z,OPD_HEX (JRZP) ; &は読み捨て
C54B 05 {X=X-1,DP=X} (DX) ; でなければ戻す
OPD_HEX:
C54C 79 C5 A0 JP OPD_HEX2
; -----
; 未使用00
; -----
C54F 00 DB

```

```

;
; 定数の直後が')'なら、その値を添字として配列要素を読む (VAR_FETCHへ)
;
-----
OPD_SUFFIX:
C550 04      {X=X+1,DP=X} (IX)
C551 24      {A=(+X)} (IXL)          ; 定数の次の文字
C552 67 31   A-&31 (CPIA)            ; ')' ?
C554 7E C5 6F JP Z,VAR_FETCH (JPZ)
C557 05      {X=X-1,DP=X} (DX)      ; 戻す
C558 DB      A↔(P) (EXAM)           ; A←値 (reg20)
C559 37      RETURN (RTN)
;
; ★未使用: CALL C540 / LP 10 / EXAM / RTN (C540は命令の途中。旧版の名残)
;
-----
C55A 78 C5 40 90 DB 37              DB
;
; 変数処理: 文字A~ZをC686+nに変換して値を読む
; ★自己書き換え: 番地下位をC575(直後のLIDPの下位オペランド)にMVDMで書き込み、
; MVDMで読む。')'が続く限り、読んだ値を添字にしてもう一度読む(A) → C600+A)。
;
-----
OPD_VAR:
C560 25      {A=(-X)} (DXL)          ; 1つ戻って変数名を読む
C561 75 51   A=A-&51 (SBIA)          ; A-Z → 0-25
C563 7F C4 F1 JP C,ERR1 (JPC)        ; A未満はエラー
C566 67 1A   A-&1A (CPIA)
C568 7D C4 F1 JP NC,ERR1 (JPNC)      ; Zを超えたらエラー
C56B A0      P=&20 (LP)               ; P→reg20
C56C DB      A↔(P) (EXAM)           ; reg20←n
C56D 70 86   (P)=(P)+&86 (ADIM)      ; reg20←86h+n
VAR_FETCH:
C56F 10 C5 75 DP=&C575 (LIDP)         ; ★C575 = 下のLIDPの下位
C572 53      (DP)=(P) (MVDM)         ; reg20を書き込み
C573 10 C6 88 DP=&C688 (LIDP)         ; LIDP C6xx (書き換え済み)
C576 55      (P)=(DP) (MVDM)         ; reg20←変数/配列の値
C577 24      {A=(+X)} (IXL)
C578 67 31   A-&31 (CPIA)            ; ')'なら値を添字にしてもう一度
C57A 39 0C   JRM Z,VAR_FETCH (JRZM)
VAR_DONE:
C57C 05      {X=X-1,DP=X} (DX)      ; 戻す
C57D DB      A↔(P) (EXAM)           ; A←値
C57E 37      RETURN (RTN)
;
; 未使用00
;
-----
C57F 00      DB
;
; END / プログラム終端(FF) ... BASIC へ普通に戻る
;
-----
END_PROG:
C580 79 C2 DD JP RET_BASIC
;
; 未使用RTN
;
-----
C583 37      DB
;
; エラー終了: BASICの文脈に付け替え、スタック上の戻り番地の下位を+1してから
; キャリーを立ててBASICへ戻る。CALL文の戻り先51CCはRC(キャリークリア)なので、
; +1して51CDに戻るとキャリーが残り、BASICはエラー(コードはreg14)として扱う。
;
-----
ERR_EXIT:
C584 78 C2 E4 CALL FIX_BASIC_CTX
C587 02 58   A=&58 (LIA)
C589 32      R=A (STR)               ; R←58
C58A 5B      POP                      ; 戻り番地下位
C58B 42      A=A+1 (INCA)            ; +1
C58C 34      PUSH
C58D D0      SET C (SC)              ; c=1: エラーを知らせる
C58E 37      RETURN (RTN)            ; BASICへ
;
; BREAK: BRK キーが押されていたら BASIC の文脈に付け替えて戻る("BREAK IN 行番号")
;
-----
BRK_EXIT:
C58F 78 C2 E4 CALL FIX_BASIC_CTX
C592 02 58   A=&58 (LIA)
C594 32      R=A (STR)               ; R←58

```

```

C595 95      P=&15 (LP)
C596 02 30    A=&30 (LIA)                ; reg15←30 (BASICの状態フラグ。BREAK扱い)
C598 DB      A↔(P) (EXAM)
C599 37      RETURN (RTN)                ; BASICへ
;
; FOR カウンタ reg1E を 0 にして INIT へ(NEXT が先に来てても素通りするのはこのため)
;
INIT_FOR:
C59A 9E      P=&1E (LP)                  ; reg1E(FORカウンタ)
C59B 60 00    (P)=(P).AND.&00 (ANIM)      ; ←0
C59D 79 C2 CA JP INIT
;
; 定数処理: 16進2桁
; ★先に2桁目(下位)を読んで判定し、DXLで1桁目(上位)に戻る。
; 2桁目が16進でなければ定数ではない → 1桁目を変数名として扱う(OPD_TO_VAR)。
; '0'-'9'(40h-49h)→0-9, 'A'-'F'(51h-56h)→40h引いて11h-16h→さらに7引いて0Ah-0Fh
; そのため"AB" "FE"のように英字2文字も定数になり、"A=A+1"は'1'を変数と見て文法エラーになる。
;
OPD_HEX2:
C5A0 04      {X=X+1,DP=X} (IX)
C5A1 24      {A=(+X)} (IXL)              ; 2文字目(下位桁)を先に読む
C5A2 67 57    A-&57 (CPIA)
C5A4 2A 28    JRP NC,OPD_TO_VAR (JRNCP) ; 57h以上→16進でない
C5A6 67 40    A-&40 (CPIA)
C5A8 3A 24    JRP C,OPD_TO_VAR (JRCP)    ; 40h未満→16進でない
C5AA 75 40    A=A-&40 (SBIA)
C5AC 67 0A    A-&0A (CPIA)                ; 0-9?
C5AE 3A 03    JRP C,HEX_LO (JRCP)
C5B0 75 07    A=A-&07 (SBIA)              ; 'A'-'F'の補正
HEX_LO:
C5B2 A0      P=&20 (LP)                  ; reg20←下位桁
C5B3 DB      A↔(P) (EXAM)
C5B4 25      {A=(-X)} (DXL)              ; 1文字目(上位桁)に戻る
C5B5 67 57    A-&57 (CPIA)
C5B7 7D C4 F1 JP NC,ERR1 (JPNC)          ; 16進でなければ文法エラー
C5BA 4D      NOP (NOPW)                  ; 詰め物
C5BB 67 40    A-&40 (CPIA)
C5BD 7F C4 F1 JP C,ERR1 (JPC)
C5C0 75 40    A=A-&40 (SBIA)
C5C2 67 0A    A-&0A (CPIA)
C5C4 3A 03    JRP C,HEX_HI (JRCP)
C5C6 75 07    A=A-&07 (SBIA)
HEX_HI:
C5C8 58      AH↔AL (SWP)                 ; 上位桁へ
C5C9 44      (P)=(P)+A (ADM)              ; reg20+=上位桁<<4
C5CA 79 C5 50 JP OPD_SUFFIX              ; ')'の確認と値の取り出しへ
OPD_TO_VAR:
C5CD 79 C5 60 JP OPD_VAR                  ; 2文字目が16進でない→変数
;
; C5D0-C5FF: COM$ 用ユーザー領域(48バイト)
;

```

参考文献

対象の記事

Fan_PC-1251 「PC-インタープリタ」、PiO 1986 年 8 月号、pp.169–171、工学社。

命令の書き方と当時の解説

「ポケコンファンに贈る PC-1250/1251 マシン語講座」1～3、POPCOM 1984 年 9 月号 pp.65–70、10 月号 pp.104–109、11 月号 pp.110–114、小学館。

私が初めてマシン語を学んだのがこの講座だった。シャープの英文マニュアルをもとにした命令表、レジスタ、フラグ、入出力ポート、メモリマップの解説があり、本書の BASIC 風命令記号 (1.2 節) と、命令の動作やサイクル数の照合に使った。

命令表

utz82, SC61860-Instruction-Set. <https://github.com/utz82/SC61860-Instruction-Set>

エミュレータ

MAME, src/devices/cpu/sc61860. <https://github.com/mamedev/mame>

matsumo, PockEmul. <https://github.com/matsumo/PockEmul>

小笠原博之, sc61860emu(1994 年). <https://github.com/hiroog/sc61860emu>

PC-インタープリタを読む

976 バイトの SC61860 マシン語を逆アセンブルする

発行日	2026 年 9 月 24 日 初版
	2026 年 9 月 30 日 訂正版 (誤りの訂正のみ)
著者	おりぐち
発行	おりぐち
解析支援	Claude(Anthropic)

解析の対象: Fan_PC-1251(https://x.com/pio1986_10) 「PC-インタープリタ」 PiO 1986 年 8 月号 pp.169-171(工学社)。本書に収録した対象プログラム (記事のダンプリストと、その逆アセンブル) の著作権は原作者に帰属します。

参 照 し た 資 料: POPCOM 1984 年 9~11 月号 「PC-1250/1251 マシン語講座」 (小学館)、utz82/SC61860-Instruction-Set、MAME(sc61860, pocketc)、PockEmul、hi-roog/sc61860emu(oldsce)。命令の BASIC 風命令記号は POPCOM の講座によります。本書の内容は、エミュレータ上での確認にもとづくもので、実機での動作を保証するものではありません。

本書の文章と図の無断転載を禁じます。